Geometry and Learning under Group Symmetries

Licentiate Thesis in Mathematics

Yacoub Hendi

Supervised by
Magdalena Larfors
Daniel Persson
Georgios Dimitroglou Rizell

Department of Mathematics
Uppsala University
Uppsala, Sweden

February 2026

This thesis is a comprehensive summary of the following two works:

1. Yacoub Hendi, Magdalena Larfors, and Moritz Walden, *Learning group invariant Calabi–Yau metrics by fundamental domain projections*, Mach. Learn.: Sci. Technol. 6 015050.

2. Yacoub Hendi, Daniel Persson, and Magdalena Larfors, *The Geometry of Polynomial Group Convolutional Neural Networks* (a manuscript, work in progress).

MACHINE LEARNING
Science and Technology

PURPOSE-LED PUBLISHING™

Part of the MACHINE LEARNING Series

PAPER • OPEN ACCESS

Learning group invariant Calabi–Yau metrics by fundamental domain projections

To cite this article: Yacoub Hendi *et al* 2025 *Mach. Learn.: Sci. Technol.* **6** 015050

View the article online for updates and enhancements.

You may also like

- Exact instantons via worldline deformations
A Akal

- Constraints on axion-like particles with different magnetic field models from the PKS 2155–304 energy spectrum
Jia Bu and Ya-Ping Li

- Fast and accurate 3D FEM model for electromagnetic simulations of no-insulation HTS coils based on polygon-anisotropic-resistivity
Zhuoyan Zhong, Wei Wu and Zhijian Jin

This content was downloaded from IP address 130.238.249.151 on 05/02/2026 at 17:09

IOP Publishing Mach. Learn.: Sci. Technol. 6 (2025) 015050 https://doi.org/10.1088/2632-2153/adb4bb

MACHINE LEARNING Science and Technology

PAPER

Learning group invariant Calabi–Yau metrics by fundamental domain projections

Yacoub Hendi*, Magdalena Larfors and Moritz Walden

Department of Physics and Astronomy & Department of Mathematics, Uppsala University, Lägerhyddsv. 1, SE-75120 Uppsala, Sweden
* Author to whom any correspondence should be addressed.

E-mail: yacoub.hendi@math.uu.se, magdalena.larfors@physics.uu.se and moritz.walden@physics.uu.se

Keywords: geometric deep learning, string theory, canonicalization, fundamental domains, Calabi-Yau manifolds, discrete symmetries, invariant neural networks

OPEN ACCESS

RECEIVED
11 September 2024

REVISED
5 February 2025

ACCEPTED FOR PUBLICATION
11 February 2025

PUBLISHED
21 February 2025

Original Content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence.

Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.

Abstract

We present new invariant machine learning models that approximate the Ricci-flat metric on Calabi–Yau (CY) manifolds with discrete symmetries. We accomplish this by combining state of the art models for predicting such metrics, based on the so-called ϕ-model of the `cymetric` package, with non-trainable, G-invariant, canonicalization layers that project the ϕ-model's input data (i.e. points sampled from the CY geometry) to the fundamental domain of a given symmetry group G. These G-invariant layers are easy to concatenate, provided one compatibility condition is fulfilled, and combine well with both standard and spectral versions of the ϕ-model. Through experiments on different CY geometries, we find that, for fixed point sample size and training time, canonicalized models give slightly more accurate metric approximations than the standard ϕ-model. On highly symmetric spaces, we also observe significantly faster convergence upon training. The method may also be used to compute the Ricci-flat metric on smooth CY quotients. We demonstrate this aspect by experiments on a smooth $\mathbb{Z}_5^2$ quotient of a 5-parameter quintic CY manifold.

1. Introduction

String compactifications lie at the heart of many formal and phenomenological studies of string theory. By compactifying some of the spatial dimensions of the space-time probed by string theory one can construct models that match observations from particle physics and cosmology, or discover how different limits of string theory are related by duality. The topology and geometry formed by the compactified dimensions are key to these constructions, and there is an interesting interplay between physics and geometric constraints. Among the compact spaces, Calabi–Yau manifolds provide a sweet spot; their mathematics is highly non-trivial, yet examples can be constructed in the thousands.

Calabi–Yau (CY) manifolds are complex, Kähler manifolds, with vanishing first Chern class; by the Calabi–Yau theorem these spaces admit a Ricci-flat metric, which is unique in a given Kähler class. Yau [1] has proven that the Kähler form J associated to the Ricci-flat CY metric g_{CY} satisfies a complex Monge-Ampère (MA) equation, which it reads, in n complex dimensions,

$$J \wedge J \wedge \ldots \wedge J = \kappa\, \Omega \wedge \overline{\Omega}\,, \tag{1}$$

where we wedge n copies of J on the left hand side, Ω denotes the unique holomorphic top form on the CY manifold, and κ is a complex constant. There is no known analytic solution to this equation for $n > 1$[1].

[1] In one complex dimension, the CY is a torus, which admits a flat metric. In 2D, i.e. for a K3 surface, a duality argument has been proposed to determine the Ricci-flat metric [2, 3].

© 2025 The Author(s). Published by IOP Publishing Ltd

Lacking an analytic solution to a partial differential equation (PDE), one may try to find numerical solutions. This is indeed possible, owing to the fact that, since J is related to any reference Kähler form J' via

$$J = J' + i\partial\bar{\partial}\phi \ , \tag{2}$$

the MA equation can be rewritten as a second order PDE for a real function ϕ. One approach to solve the PDE for ϕ, is to expand ϕ in degree-k polynomials on the CY, an idea that goes back to Tian [4], and then solve for the expansion coefficients. This can be done using a fixed-point iteration scheme, known as the Donaldson algorithm, [5–8], or via functional minimization [9–11]; collectively, the metrics obtained in this way are called *algebraic* metrics. Since the polynomial, or *spectral*, basis is complete in the limit $k \to \infty$ [5, 9], this method will indeed give the unique Ricci-flat metric in this limit. However, for most CY spaces, the spectral basis dimension grows rapidly with k, implying that practical applications are limited to rather small k[2]. For symmetric CY geometries, using an invariant spectral basis, of smaller dimension, allows us to compute the metric at larger k (in fact, the most accurate approximations of CY metrics are obtained on highly symmetric Fermat quintics [9])[3].

With the advent of machine learning (ML), novel opportunities, based on neural networks, have arisen to approximate CY metrics. Since neural networks are universal approximators [12–14], they can learn any function; thus ϕ, or why not the Ricci-flat CY metric directly, may be learned by an ML model. Developing machine learning methods for CY geometry has been an active field of research in later years [15–27]. A major practical gain of ML methods is that they are less dependent on expansions in the spectral basis. While ML methods may be used in combination with the spectral expansion of ϕ, they can also make away completely with this crutch. As such, the ML methods remain computationally feasible on generic CY manifolds, as evidenced in references [21, 22, 27].

Admittedly, the accuracy of ML algorithms has been observed to improve on more symmetric spaces [18, 19, 24, 27], and the training time (for similar performance) is slightly longer on non-symmetric spaces [27]. These differences, which are small compared to the effects seen in pre-ML methods, indicate that the ML models learn the symmetries of the data they are trained on. Symmetries have also been used to test the accuracy of ML predictions of CY metrics [17]. However, discrete symmetries have, so far, not played any significant role in the design of ML models for CY metrics. In this paper, we will perform a first exploration of this topic, using novel methods to construct group invariant ML models[4].

The motivation for our study is two-fold. First, as we will review below, in the ML literature there is ample evidence that for symmetric data, equivariant and invariant models outperform networks that are agnostic about symmetries in the data. The Geometric Deep Learning program [28, 29] shows how symmetries explain the success of state-of-the-art ML models such as CNNs, GNNs, and Transformers; reference [30] analyse the CNNs using the concept of covariance in theoretical physics. Formally, it has been proven that group invariant neural networks are universal approximators for group invariant functions [31], thus symmetry-enforcing ML models are as expressive as the symmetry-agnostic ones.

Secondly, from the perspective of string theory, there is phenomenological motivation to study non-simply connected CY geometries, obtained by quotienting a CY manifold by a freely acting discrete symmetry G. G-invariant ML models provide a new way to compute the Ricci-flat metrics of such quotients. This problem has been explored using Donaldson's algorithm in the past [8], with positive effects on the accuracy of the metric approximation. reference [8] also explains the technical subtleties associated with the creation of a group invariant spectral basis; this provides further motivation to explore invariant ML methods, as do recent application of ML methods to the computations of heterotic Yukawa couplings on CY manifolds with discrete symmetries [32, 33].

Present work

In this work, we will design group invariant ML models that provide approximations of the Ricci-flat metric of CY manifolds. Our work is motivated by the importance of symmetries in the context of spectral methods for CY metrics, and the role symmetries play in leading ML models.

For our studies, we will explore CY geometries created as complete intersections in complex projective spaces, so-called CICYs [34]. We will use the `cymetric` package, introduced in [21, 22], which allows us to sample points on such spaces, and provide several ML model architectures that either predict the function ϕ, or the metric directly. In this work, we will limit our attention to the ϕ-model. To create group invariant

[2] For a CY hypersurface in $\mathbb{CP}^N$, the dimension of the polynomial basis scales like k^{2N} for large k [10].

[3] However, constructing the invariant basis sometimes comes at a non-trivial computational cost, as we will discuss in section 6.

[4] In ML methods based on spectral basis, such as [16, 23] and [18, 19], one can restrict to group invariant elements of the basis; to the knowledge of the authors this has not been explored to any greater degree.

ϕ-models, we will include non-trainable layers into the neural network architecture. These layers pre-process the the input data of the ϕ-model, and ensure that it is G-invariant by projecting to a fundamental domain of the group G. This idea, which is often referred to as *canonicalization*, has recently been proposed in [35, 36]. Our main contribution is a comparison of the performance and accuracy of the canonicalized ML models with the standard ϕ-model. We also discuss the similarities and differences of the canonicalization approach with the spectral networks suggested by [24], and how canonicalization can be combined with spectral ϕ-models. Finally, we study how canonicalized ML models can be used to compute the metric on a non-simply connected CY manifold[5].

This paper is organised as follows. In section 2, we briefly review related work on machine learned, Ricci-flat CY metrics, and outline the key approaches to group invariant ML models. Section 3 describes CY geometry and the ML model we will use for our experiments. We also recall how the spectral layer is constructed. In section 4, we present our main contribution: how to construct canonicalization layers for the symmetry groups relevant for CY point samples; experiments using these techniques are presented and discussed in section 5. Some technical aspects of our analysis are described in appendices. In section 6, we apply canonicalization layers to the computation of Ricci-flat CY metrics on smooth quotients of a 5-parameter quintic CY. Conclusions and discussions are given in section 7.

2. Related work

The past five years have witnessed a flurry of research activities on ML methods for Ricci-flat CY metrics [15–25], and there are several recent examples in the literature on how these ML methods may be used to address open questions in string theory [25, 32, 33, 37, 38]. Since the Ricci-flat CY metric is unknown, all ML models for CY metrics rely on some sort of semi-supervised learning, where a custom loss function is used to ensure that the learned metric is Ricci-flat. This is in reminiscent of Physics Informed Neural Networks (PINNs) [39]; in the parlance of PINNs, the Ricci-flatness is encoded as a soft constraint, meaning that it will only be approximatively enforced by minimizing an appropriate loss function. In contrast, a hard constraint is implemented directly into the network architecture and thus does not need to be learned.

To date, three open-source ML libraries have been developed. While we will only explore one of these libraries, the key features of the other packages may also be of interest to the reader, and are therefore given here

- The `MLgeometry` package was developed for references [18, 19, 38]. This package introduces the bihomogeneous (holomorphic) neural networks where the layers are bihomogeneous (holomorphic) functions of the ambient space coordinates for the CY manifold, with quadratic non-linear activation functions. These layers encode the spectral basis of the algebraic CY metrics, and the learned metric is automatically globally defined and Kähler (i.e. these are hard constraints). The Ricci-flatness condition is imposed softly via a loss function.
- The `cymetric` package, presented in references [21, 22] and further tested in reference [40], instead encodes the CY metric, or alternatively the scalar function ϕ defined in equation (2), directly in a dense, shallow, fully connected neural network (typically with ReLU or GELU activation functions). Custom loss functions are used to enforce that the learned metric is globally defined, Kähler, Ricci-flat, and restricted to the Kähler class selected when starting the training[6]. The package is state of the art as it has full functionality for point sampling and network training across established databases of example CY geometries. In this paper, we will be using the ϕ-model of the `cymetric` package, which approximates the function ϕ. This model is the best performing in the `cymetric` package [21, 22]. We will elaborate further on said model in section 3.
- the `cyjax` package [23] is a spectral method. The ML model learns algebraic CY metrics; the neural net encodes an Hermitian coefficient matrix, which, when combined with the spectral basis, provides an approximation to the Ricci-flat metric. This automatically provides a globally defined, Kähler metric; the Ricci-flatness condition is imposed via a loss function.

In reference [24], it was proposed that the ϕ-model of the `cymetric` package can be augmented by a non-linear, non-trainable *spectral layer*, where the input data is passed to the $k = 1$ spectral basis, and then fed to the dense, trainable layers of the model. The benefit of this layer is that the predicted metric is automatically globally defined and Kähler. Thus, some of the soft constraints of the ϕ-model become hard constraints with the addition of a spectral layer. In reference [24], this was shown to be of essence in learning

[5] The code for the canonicalization layers can be found in https://github.com/jake997/invariant_layers_cymetric.

[6] The motivation for the soft encoding of all mathematical constraints on the metric, was to make the package generalize, at little cost, to other geometric constructions, beyond the CY regime.

the Ricci-flat metric on singular, or nearly singular, CY manifolds. The spectral layer is, from a machine learning perspective, a data pre-processing step; as we will discuss below it shares some features with the ideas explored in this paper.

In this paper, we will be interested in predicting a function $\phi : X \to \mathbb{R}$, where X is a CY manifold. Working locally, we may decompose X into patches which are isomorphic to $\mathbb{R}^n$. The prediction of a function

$$f : \mathbb{R}^n \to \mathbb{R}$$

is a standard task in machine learning. When the function is invariant/equivariant[7] under a certain group action, invariant (and equivariant) machine learning models are of interest. This topic is extensively studied. An excellent starting point for studies of the mathematical foundations of invariant and equivariant ML models can be found in the book [29], which also provides a comprehensive list of references to this field of data science.

There are different approaches to the construction of invariant machine learning models. Following Yarotsky [31], see also [35], one may distinguish between *symmetrization-based* and *intrinsic* approaches. In the first approach, one may, for example, construct G-invariant models by augmenting the training data by G-transformed copies with the same label as the original data point. The network will then learn that data points related by G-transformations give the same output, as required by invariance. When applied to unseen data, a network trained on augmented data will perform better than a network trained on non-augmented data. As discussed in reference [41], from a mathematical point of view, and under exact invariance, data augmentation is equivalent to averaging over orbits of the group G; it can thus be proven that this reduces the variance of the prediction of a model.

Intrinsic approaches instead produce invariant networks by imposing restrictions on the network design. CNNs, which are equivariant under translations, are early incarnations of this idea [42, 43]. Other network designs that are invariant under group transformations include [44–48]; these and related ideas are further discussed in [29] and the review [49]. Common to these designs is that in- or equivariance is implemented by weight-sharing in some layer of the network.

Restricting the trainable weights of some layer in the network is not the only intrinsic way to achieve invariant/equivariant layers. Yarotsky has shown, using classical polynomial invariant theory, that an extra polynomial layer which is invariant/equivariant and complete, results in an invariant/equivariant network [31]. This is particularly useful for finite groups, where complete invariant layers can be constructed from a finite number of polynomial invariants. For continuous groups, invariant polynomial layers can also be constructed, but, lacking completeness, one cannot prove that they are universal invariant approximators [31]. The spectral layer [24] (described briefly above and in detail below) is, up to rescaling by the norm, an incarnation of such an invariant layer for the continuous group $\mathbb{C}^*$.

Of key interest to us is the alternative intrinsic approach proposed in [35, 36]. Here, a pre-processing step is used to put the data into a canonical form, by projection to a fundamental domain of the symmetry group. The canonicalization can be achieved in different ways, and we will describe this method in more detail below. Importantly, the method works for both continuous and discrete groups. It is also good to note that, in contrast to weight-sharing schemes, that methods based on polynomial invariants or canonicalization are not restricted to models using neural networks. Since the concatenation of an invariant map with any other map preserves invariance, such data pre-processing steps can be used to impose invariance on virtually any type of ML model, for example the packages developed for machine learning of CY metrics.

3. Machine learning Ricci-flat CY metrics

In this section, we review some aspects of the construction of examples of CY manifolds, and their symmetries. A more detailed discussion of the symmetries is relegated to appendix A. We then describe the key points of the `cymetric` package, and the ML methods we employ to learn CY metrics. For more details on the `cymetric` package, we refer to [21, 22].

3.1. CY construction

For our experiments, we need explicit realizations of CY geometries. In this paper, we will restrict our studies to the Complete Intersection Calabi-Yau (CICY) manifolds [34, 50][8]. These are constructed as the zero loci

[7] By this we mean, that for $g \in G$, where G is the considered symmetry group, f transforms as $f(g \cdot x) = \rho(g)f(x)$, where ρ is a representation of G. In the invariant case, ρ corresponds to the trivial representation.

[8] It would be interesting to extend our studies to CY manifolds in toric ambient spaces. The `cymetric` package has point generators also for such spaces, and the methods we develop here can be adapted to this setting. We leave this generalisation for future work.

of a set of homogeneous polynomials p_i, $i = 0, \ldots, N$, in an ambient space $\mathbb{CP}^{n_1} \times \cdots \times \mathbb{CP}^{n_N}$, such that $\dim(CY) = n_1 + \cdots + n_N - N$. A prime example is the Fermat quintic, defined by the solutions of

$$\sum_{i=0}^{4} z_i^5 = 0 \tag{3}$$

in $\mathbb{CP}^4$. CICY manifolds are often described in terms of configuration matrices, which in the case of the quintic is [4|5]. This class of CY manifolds is fully classified, and a detailed list of their topological properties can be found in [51].

CICY manifolds may, at certain subloci in their complex structure moduli space, admit discrete symmetries[9]. These are manifest as invariances of the defining polynomials of the CICY. For example, the defining equation of the Fermat quintic (3) is invariant under permutations of the coordinates, multiplications of the coordinates by 5th roots of unity (up to a global multiplication by the same root, which is removed by the scaling invariance of the homogeneous coordinates), and complex conjugation; the full symmetry group is $\mathbb{Z}_5^4 \rtimes (S^5 \times \mathbb{Z}_2)$ [8]. The freely acting symmetries of CICY manifolds have been classified in [52], non-freely acting symmetries are classified in [53], and symmetries of CICY quotients are classified in [54]. The invariance of the defining polynomials translates into an isometry of the Ricci-flat CY metric (see appendix A for a proof).

We want to learn the Ricci-flat metric on some CICY manifold X with dimension n. Starting from a reference Kähler form J', we know, by the CY theorem, that there exists a unique Kähler form J, in the same cohomology class as J', which gives a Ricci-flat metric.

For simplicity, let us discuss the case where X is realized as one hypersurface in one projective space. All our statements generalize to the case of complete intersections of hypersurfaces, so this is just for ease of presentation. Let $\iota : X \hookrightarrow \mathbb{CP}^{n+1}$ be the inclusion of X in its ambient space, and J' be the pullback of the Kähler form ω associated to the Fubini-Study metric on $\mathbb{CP}^{n+1}$ to X, i.e. $J' = \iota^\star \omega$. Then we want to learn a real function $\phi : X \to \mathbb{R}$ such that

$$J = J' + i\,\partial\overline{\partial}\phi \tag{4}$$

is Ricci-flat. We use the `cymetric` library[10] [21, 22] for learning said metric, including the ϕ-*model*, which encodes equation (4).

3.2. The ϕ-model of the cymetric package

The `cymetric` package consists of point generators, that create samples of points on the CY manifold, and a number of machine learning models that take the coordinates of the sampled points as input, and gives some data related to the Ricci-flat CY metric as output. In the ϕ-model, the output is a numerical estimate of the real function ϕ, defined in equation (4).

Since data pre-processing is a theme of this paper, we need to carefully describe the input data of the ϕ-model. As mentioned, this data represents points on the CY manifold, as constructed by the point generators of the `cymetric` package; these routines are discussed in depth in reference [22]. For simplicity, we continue to focus on the case where X is a hypersurface in $\mathbb{CP}^{n+1}$. Points in the CY n-fold X are represented by homogeneous coordinates, z, of the ambient space $\mathbb{CP}^{n+1}$, and are by default repackaged, by the `cymetric` point generators, as a real tuple

$$(\Re(z_0) \quad \ldots \quad \Re(z_{n+1}) \quad \Im(z_0) \quad \ldots \quad \Im(z_{n+1})) \in \mathbb{R}^{2(n+2)},$$

where the homogeneous scaling invariance of $\mathbb{CP}^{n+1}$ has been used to ensure that the input data take values in the range $[0, 1]$. In accordance with the philosophy of this paper, we will still refer to the network as a map $f : X \to \mathbb{R}$.

In order to train the neural network, we rely on loss functions encoding the relevant mathematical constraints. In the `cymetric` package, these constraints are encoded in five custom loss functions. First, we want to ensure Ricci-flatness. As mentioned in the introduction, this can be phrased as requiring J to obey the Monge–Ampère equation, which in 3 dimensions reads

$$J \wedge J \wedge J = \kappa\,\Omega \wedge \overline{\Omega}, \tag{5}$$

[9] We define CY 3-folds to have strict $SU(3)$ holonomy; consequently, the spaces we study do not admit continuous isometries.
[10] https://github.com/pythoncymetric/cymetric.

where κ is some complex constant and Ω is the no-where vanishing holomorphic $(3,0)$-from[11]. With this at hand, the first loss function is defined as

$$\mathcal{L}_{\mathrm{MA}} = \left\| 1 - \frac{1}{\kappa} \frac{J \wedge J \wedge J}{\Omega \wedge \overline{\Omega}} \right\|_1 , \tag{6}$$

where $\|\cdot\|_1$ denotes the L_1 norm[12].

Furthermore, we want to ensure that ϕ is a global function. Hence we define the transition loss

$$\mathcal{L}_{\mathrm{transition}} = \frac{1}{d} \sum_{(s,t)} \left\| \phi^{(t)} - \phi^{(s)} \right\|_1 , \tag{7}$$

where (s,t) denotes a sum over all distinct pairs of patches and d denotes the number of patch transitions.

In the `cymetric` package, three more constraints are encoded as custom loss functions $\mathcal{L}_i$: the Kähler constraint $\mathrm{d}J = 0$ gives $\mathcal{L}_{\mathrm{dJ}}$, preserving the Kähler class is enforced by $\mathcal{L}_{\mathrm{Kclass}}$[13] and the vanishing Ricci-scalar is given by $\mathcal{L}_{\mathrm{Ricci}}$. The total loss function is thus given by

$$\mathcal{L} = \alpha_1 \mathcal{L}_{\mathrm{MA}} + \alpha_2 \mathcal{L}_{\mathrm{dJ}} + \alpha_3 \mathcal{L}_{\mathrm{transition}} + \alpha_4 \mathcal{L}_{\mathrm{Ricci}} + \alpha_5 \mathcal{L}_{\mathrm{Kclass}} . \tag{8}$$

Here, α_i are tuneable hyperparameters of the model that are set by the user. In our experiments, we use the default values for the `cymetric` losses in the ϕ-model; thus the Kähler and Ricci losses are disabled and $\alpha_i = 1, i \in \{1,3,5\}$. We refer the reader to [21, 22] for more discussions of the loss functions.

3.3. Spectral layers

The spectral network, introduced in [24], repackages the input data of the ϕ-model of the `cymetric` package in a form that is manifestly invariant under homogeneous rescalings of the ambient space coordinates. The motivation for this feature engineering step is to ensure that the learned function ϕ is globally well-defined. Recall that this is only enforced 'softly', by a loss function, in the `cymetric` package. As a consequence, the network may predict line bundle sections rather than globally defined functions. An observed ill consequence of this is the discrepancy of topological quantities computed with the predicted Ricci-flat metric [24], which becomes particularly noticeable for singular, or nearly singular, CY spaces.

It may nevertheless appear surprising that such feature engineering is needed. The homogeneous rescaling is not a symmetry of the CY manifold; this is a complex, n-dimensional space with n holomorphic coordinates. However, as we have described in the previous section, the input data of the `cymetric` models is not an n-dimensional complex tuple. Rather, for a CICY with ambient space $\mathbb{CP}^{n_1} \times .. \times \mathbb{CP}^{n_k}$, the point generators of `cymetric` produces $\sum_i 2(n_i+2)$-dimensional real input data tuples (with entries in the range $[0,1]$). In this pre-processing the points on the CY space are separated into coordinate patches (that overlap but for zero measure sets). The training is then done locally, using the transition loss function to impose that the function matches on patch overlaps. It is an empirical fact that the transition loss function is around 10^{-3} for standard ϕ-models (trained for 10 epochs on the Fermat quintic). As we will see in section 5, with the addition of a spectral layer in the network, the transition loss function is of the order 10^{-9} already at initialization of the network, which is negligible within numerical precision and allows us to treat the prediction of ϕ as a function.

The effect of the spectral layer is thus to impose invariance under homogeneous rescalings. Following reference [24] (see also [37]), we have encoded the layer as follows[14]. For a CY with ambient space $\mathbb{CP}^n$, the spectral layer is a map

$$(z_0, \ldots, z_n) \mapsto \begin{pmatrix} \frac{z_0\bar{z}_0}{|z|^2} & \frac{z_0\bar{z}_1}{|z|^2} & \cdots & \frac{z_0\bar{z}_n}{|z|^2} \\ \frac{z_1\bar{z}_0}{|z|^2} & \frac{z_1\bar{z}_1}{|z|^2} & \cdots & \frac{z_1\bar{z}_n}{|z|^2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{z_n\bar{z}_0}{|z|^2} & \frac{z_n\bar{z}_1}{|z|^2} & \cdots & \frac{z_n\bar{z}_n}{|z|^2} \end{pmatrix}$$

[11] Here J is predicted by the network and κ and Ω are computed by the point generator.

[12] The `cymetric` package can use any L_n norm for the loss functions, we use the default setting of the package.

[13] The Kähler class loss function is necessary when learning the metric on any CY manifold with $h^{(1,1)} > 1$. It reduces to a volume-preserving constraint when $h^{(1,1)} = 1$. The `cymetric` routines require such a loss since they can be applied to any CICY manifold, and CY manifolds defined as hypersurfaces in the toric ambient spaces contained in the Kreuzer–Skarke list [55].

[14] This encoding shows the relation between the spectral layers and the bihomogeneous layers of [18]; they are essentially related by a division by the norm. However, the bihomogeneous layers have trainable weights, whereas the spectral layer is non-trainable.

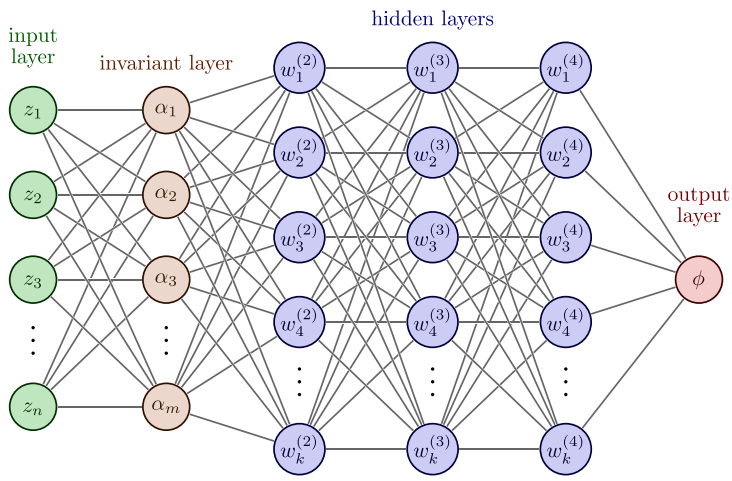

Figure 1. A dense neural network with an invariant layer. The invariant layer does not have trainable parameters. Its sole purpose is to put the input data, z_i, in a form, $\alpha_i(z)$ that is invariant under the relevant symmetry group. Hence, it always appears directly after the input layer.

where $|z|^2 = \sum_{i=0}^{n} z_i \bar{z}_i$. This is clearly invariant under

$$z \to \lambda z, \lambda \in \mathbb{C}^\star \,.$$

The entries in the spectral layer matrix corresponds to the degree-1 basis of eigenfunctions of the Laplacian on $\mathbb{CP}^4$ introduced in [37]; this is the rationale for the name of the layer. Comparing to the machine learning literature, this is almost, up to the division by the norm, an instance of the polynomial invariant layers studied by [31].

For CICY manifolds with product ambient spaces $\mathbb{CP}^{n_1} \times \mathbb{CP}^{n_2} \times \ldots \mathbb{CP}^{n_k}$ the spectral layer is implemented in a block diagonal fashion[15], with separate maps for the homogeneous coordinates associated to each $\mathbb{CP}^{n_j}$.

As it is evident from the formulae above, our spectral layer uses complex inputs. To match the real input of the `cymetric` package, we use a conversion between the real inputs to complex and then back to real. Whenever we refer to the spectral layer below, we refer to this composition.

Note that there are no trainable parameters in spectral layers. Their only effect is to map the input data to a convenient, canonical form, that is invariant under homogeneous rescalings. Since the composition of an invariant map with any other map, results in a map with the same invariance, the inclusion of a spectral layer results in an invariant machine learning model, as illustrated in figure 1.

In the next section, we will explore other types of invariant layers, that serve the purpose of making the network prediction ϕ invariant under isometries of the CY manifold.

4. Invariant metric by fundamental domain projections

In the following, we will discuss a method to construct a group invariant neural network predicting the Ricci-flat metric on some CY manifold. This will be done by projecting the input data, i.e. the tuple containing the homogeneous coordinates of the sampled points on the CY space, onto the fundamental domain of said group, following the strategy of [35, 36]. We recall that the rationale of these methods is based on the fact that the composition of a group invariant map with any other map, results in a map with the same invariance. Thus, by including a non-trainable, G-canonicalization layer, as introduced below, before the dense layers of the ϕ-model, we get a group invariant network. An illustration of this model architecture is given in figure 1.

We consider a neural network $f : X \to \mathbb{R}$ for some CY manifold X. Given some function $h : X \to X$, satisfying

$$h(g \cdot x) = h(x) \,, \qquad \forall x \in X, \forall g \in G, \tag{9}$$

[15] Note, that these matrices are never explicitly constructed in our implementation. Instead, the input is directly computed as a flattened vector, where the zero entries are discarded, that can be fed into the neural network.

for a group G acting on X, one can construct a G-invariant neural network by

$$f(h(x)) = y, \qquad x \in X, y \in \mathbb{R}. \tag{10}$$

This can be achieved by defining a map $\pi : X \to G$, obeying

$$\pi(g \cdot x) = \pi(x) g^{-1}, \qquad \forall x \in X, \forall g \in G, \tag{11}$$

such that

$$h(x) = \pi(x) \cdot x. \tag{12}$$

We call the map h a *G-canonicalization* and it is easy to see that h is a G-invariant map

$$h(g \cdot x) = \pi(g \cdot x) \cdot (g \cdot x) = \pi(x)\left(g^{-1} g\right) \cdot x = \pi(x) \cdot x = h(x). \tag{13}$$

The remaining task is to define an appropriate map that obeys equation (11). We will do so by defining maps to a fundamental domain of G.

Definition 4.1 (Fundamental domain). Let G be a group acting on a topological G-set X. Then a closed subset $\mathcal{F}$ of X is called a fundamental domain of G in X if X is the union of all conjugates of $\mathcal{F}$, i.e.

$$X = \bigcup_{g \in G} g\mathcal{F}, \tag{14}$$

and the intersection of any two conjugates has no interior.

With this definition at hand, we define $h : X \to \mathcal{F}$, such that $h(g \cdot x) = h(x)$ which defines π with respect to a choice of $\mathcal{F}$. Note, that for a given choice of h and $\mathcal{F}$, π is unique.

It is important to note that the projection into a fundamental domain can give rise to discontinuities. For a given fundamental domain $\mathcal{F}$ of some group G acting on some manifold X, there exist points on X which will get mapped to the boundary of $\mathcal{F}$. Moving infinitesimally within X can then result in discontinuous jumps in $\mathcal{F}$. For the metric approximations we construct, this can lead to problems computing global properties, like the volume which require integration over the entire manifold. We will talk about this further in section 5 and the conclusion.

4.1. Examples

In the following examples, we will consider symmetries on the Fermat quintic, whose defining equation was given in equation (3). Let $f \circ h : X \to \mathbb{R}$ be a neural network learning the function $\phi(z)$ in the Ricci-flat metric $g_{\mathrm{CY}}(z) = g_{\mathrm{FS}}(z) + \partial\bar{\partial}\phi(z)$. Let $\mathbf{z} \in \mathbb{C}^5$ encode z as a vector and let $\rho : G \to \mathrm{GL}(5, \mathbb{C})$ be a representation of some symmetry group G. A G-invariant network is then constructed by defining $h(\mathbf{z}) = \rho(\pi(z))\mathbf{z}$ together with an appropriate choice of $\pi(z)$. In the following, we will construct examples considering several different symmetry groups G. In each example, we specify explicitly the action of G on the ambient space, the fundamental domain $\mathcal{F}$ and the projection map h on $\mathcal{F}$.

4.1.1. Homogeneous rescalings

Consider the set of homogeneous rescalings of the homogeneous coordinates of the embedding space of the quintic, $\mathbb{CP}^4$:

$$[z_0 : z_1 : z_2 : z_3 : z_4] \sim \lambda [z_0 : z_1 : z_2 : z_3 : z_4], \text{ for } \lambda \in \mathbb{C}^\star.$$

The group $G = \mathbb{C}^\star$ preserves the equivalence classes of the input data

$$(\Re(z_0) \quad \ldots \quad \Re(z_4) \quad \Im(z_0) \quad \ldots \quad \Im(z_4)) \in \mathbb{R}^{10}$$

of the ϕ-model. Hence, we require ϕ to be invariant under actions of $\mathbb{C}^\star$.

We define the fundamental domain

$$\mathcal{F} = \left\{ [z_0 : z_1 : z_2 : z_3 : z_4] \in \mathbb{CP}^4 \,\middle|\, \forall i : |z_i| \leqslant 1, \exists i : z_i = 1 \right\}, \tag{15}$$

and the projection map h_{hom} to be

$$h_{\mathrm{hom}} : [z_0 : z_1 : z_2 : z_3 : z_4] \mapsto \left[\frac{z_0}{z_i} : \frac{z_1}{z_i} : \frac{z_2}{z_i} : \frac{z_3}{z_i} : \frac{z_4}{z_i} \right], \forall j \neq i : |z_i| \geqslant |z_j|. \tag{16}$$

In case more than one coordinate has the biggest norm, we choose to divide by the coordinate with smallest index[16].

Equation (15) thus uniquely determines the canonicalization maps h and π defined in the beginning of this section. The action transforms any input into the canonical form of $\max(|z_0|, \ldots, |z_4|) = 1$. Note that the `cymetric` input of the point generator is already of this canonical form, hence the layer has no effect. However, upon training, points are re-scaled within the transition loss to compare the prediction of ϕ in different patches. This re-scaling is inverted by $h_{\text{hom}}(\mathbf{z})$, leading to an almost vanishing transition loss.

As an example of the action of this map, consider the point

$$z = [z_0 : z_1 : z_2 : z_3 : z_4], \text{ where } \forall j \neq 3 : |z_3| > |z_j| .$$

Then $\pi(z) = 1/z_3$ and in terms $\mathbf{z} \in \mathbb{C}^5$ we have

$$h_{\text{hom}}(\mathbf{z}) = \rho(\pi(z)) \cdot \mathbf{z} = \begin{pmatrix} 1/z_3 & 0 & 0 & 0 & 0 \\ 0 & 1/z_3 & 0 & 0 & 0 \\ 0 & 0 & 1/z_3 & 0 & 0 \\ 0 & 0 & 0 & 1/z_3 & 0 \\ 0 & 0 & 0 & 0 & 1/z_3 \end{pmatrix} \begin{pmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix} = \begin{pmatrix} z_0/z_3 \\ z_1/z_3 \\ z_2/z_3 \\ 1 \\ z_4/z_3 \end{pmatrix} . \tag{17}$$

When the ambient space is a product of complex projective spaces, we apply homogeneous rescaling as described above for the coordinates of each factor, separately.

4.1.2. Permutations

Let $G = S_5$, which acts by permutation of variables. Clearly, the action of this group allows to reorder the entries of a tuple $(z_0, \ldots, z_n)$ in any way we desire, and we may select a fundamental domain by any consistent order. Thus, given a set of points $[z_0 : z_1 : z_2 : z_3 : z_4]$, we define the fundamental domain

$$\mathcal{F} = \left\{ [z_0 : z_1 : z_2 : z_3 : z_4] \in \mathbb{CP}^4 \,\middle|\, |z_0| \geqslant |z_1| \geqslant |z_2| \geqslant |z_3| \geqslant |z_4| \right\} . \tag{18}$$

The projection map h_{perm} thus performs the required sorting[17]. By default, if the norms are equal, h_{perm} keeps the original order.

As an example, consider the point $z = [z_0 : z_1 : z_2 : z_3 : z_4]$ where $|z_1| > |z_0| > |z_2| > |z_3| > |z_4|$, then

$$\pi(z) = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 \\ 1 & 0 & 2 & 3 & 4 \end{pmatrix} , \tag{19}$$

where we have used *Cauchy's two line notation* for $\pi(z) \in S_5$, where the first line corresponds to indices i and the second line to their permutations $\sigma(i)$. In terms of $\mathbf{z} \in \mathbb{C}^5$, we have

$$h_{\text{perm}}(\mathbf{z}) = \rho(\pi(z)) \cdot \mathbf{z} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix} = \begin{pmatrix} z_1 \\ z_0 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix} . \tag{20}$$

In the case of product ambient spaces, there may be multiple permutation invariances. Below, we will explore CY manifolds whose ambient space is $(\mathbb{CP}^n)^m$, with defining polynomials that are invariant under an S_n that acts identically on each factor. Concretely, the fundamental domain of such a diagonal S_n acting on $\mathbb{CP}^n \times \mathbb{CP}^n$ is defined to be

$$\mathcal{F} = \left\{ ([z_0 : z_1 : \ldots : z_n], [w_0 : w_1 : \ldots : w_n]) \in \mathbb{CP}^n \times \mathbb{CP}^n \,\middle|\, |z_0| \geqslant |z_1| \geqslant \ldots \geqslant |z_n| \right\} \tag{21}$$

where the restriction is only enforced on the coordinates of the first component. To project on $\mathcal{F}$, we order the coordinates in the first factor by norms and then use the same ordering to order the rest coordinates of the remaining factors. For example, the point $([z_0 : z_1 : \ldots : z_n], [w_0 : w_1 : \ldots : w_n]) \in \mathbb{CP}^n \times \mathbb{CP}^n$, such that $|z_1| \geqslant |z_0| \geqslant |z_2| \geqslant |z_3| \geqslant \ldots \geqslant |z_n|$ gets mapped under the projection to $([z_1 : z_0 : z_2 : z_3 : \ldots : z_n],$

[16] This is straightforwardly implemented in `TensorFlow` by finding the largest coordinate with `tf.math.argmax(tf.abs(complex_input))` and then dividing the input by said coordinate.

[17] Again, this is straightforwardly implemented in `TensorFlow` by obtaining the absolute values of the input with `norms = tf.math.abs(complex_inputs)` and then sorting by `tf.argsort(norms, axis = -1, direction = `DESCENDING')`.

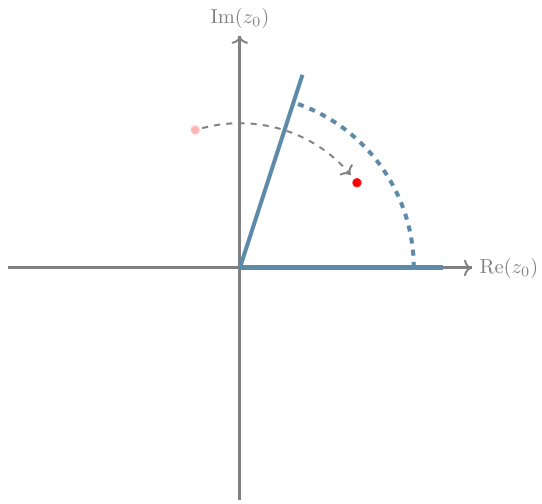

Figure 2. Under scaling by a fifth root of unity, z_0 maps into the fundamental domain $\mathcal{F}$ (first quintant) by an appropriate rotation.

$[w_1 : w_0 : w_2 : w_3 : \ldots : w_n]$). While we do not foresee any large complications, we leave the adaptation of the fundamental domain projections to ambient spaces of the type $\mathbb{CP}^m \times \mathbb{CP}^n$ with $m \neq n$, and similar generalizations, to future work.

4.1.3. Scaling by roots of unity

Let $G = (\mathbb{Z}_5)^5$ act by re-scaling each coordinate by a fifth root of unity, e.g. $z_i \to \exp(2\pi i k/5) z_i$ for some $k \in \{0, \ldots, 4\}$. Note, that different coordinates can be re-scaled by different roots simultaneously. We define the fundamental domain

$$\mathcal{F} := \{[z_0 : \ldots : z_4] \in \mathbb{CP} \mid \forall i : 0 \leqslant arg(z_i) \leqslant 2\pi/5\} ,$$

and the projection map $h_{\text{RootSc}} : X \to \mathcal{F}$ is given by[18]

$$h_{\text{RootSc}} : \left[r_0 e^{i\theta_0} : \ldots : r_4 e^{i\theta_4}\right] \to \left[r_0 \exp\left[i\left(\theta_0 \mod 2\pi/5\right)\right], \ldots, r_4 \exp\left[i\left(\theta_4 \mod 2\pi/5\right)\right]\right] . \tag{22}$$

As an example, consider the point $z = [r_0 e^{2\pi i \varphi_0} : r_1 e^{2\pi i \varphi_1} : r_2 e^{2\pi i \varphi_2} : r_3 e^{2\pi i \varphi_3} : r_4 e^{2\pi i \varphi_4}]$ where $\varphi_i < 1/5 \; \forall i \neq 0$ and $\varphi_0 = 3/10$. We have for $\mathbf{z} \in \mathbb{C}^5$

$$h_{\text{RootSc}}(\mathbf{z}) = \rho(\pi(z))\mathbf{z} = \begin{pmatrix} \exp(2\pi i 4/5) & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} r_0 e^{2\pi i 3/10} \\ r_1 e^{2\pi i \varphi_1} \\ r_2 e^{2\pi i \varphi_2} \\ r_3 e^{2\pi i \varphi_3} \\ r_4 e^{2\pi i \varphi_4} \end{pmatrix} = \begin{pmatrix} r_0 e^{2\pi i 1/10} \\ r_1 e^{2\pi i \varphi_1} \\ r_2 e^{2\pi i \varphi_2} \\ r_3 e^{2\pi i \varphi_3} \\ r_4 e^{2\pi i \varphi_4} \end{pmatrix} . \tag{23}$$

We illustrate the projection in figure 2.

4.2. Combining invariant layers

To achieve metric predictions that are invariant under a (semi)-direct product of groups, as is the case for the Fermat quintic, we would like to construct ML models that are invariant under several groups. This can be attained by adding the G-canonicalization layers in sequence to the network. However, we must then make sure that the canonicalizations satisfy a non-trivial condition: Let $G = G_1 \ltimes G_2$ act on a space X and let h_i be a G_i-canonicalization for $i = 1, 2$. Then $h := h_2 \circ h_1$ defines a G-canonicalization if and only if h_1 descends to a well defined map on the orbits of G_2, namely if $h_1 : X/G_2 \to X/G_2$ is well defined. Note that this condition on h_1 is weaker than h_1 being equivariant with respect to G_2.

To make this concrete, let us return to the metric on the Fermat quintic X defined by (3), which is invariant under action of the group $G = S_5 \ltimes \mathbb{Z}_5^5$. For later reference, we note that the action of this group is

[18] We implement this projection in `TensorFlow` by finding the angles with `angles = tf.math.angle(complex_input)` and then applying the modulo operation with`tf.math.floormod(angles, 2*np.pi/5.)`.

not free. Let h_{perm} be the S_5-canonicalization defined by (18), and let h_{RootSc} be a $\mathbb{Z}_5^5$-canonicalization as defined in (22). Then $h_{\text{RootSc}} \circ h_{\text{perm}}$ is a G-canonicalization, since for any $r \in \mathbb{Z}_5^5$ and $z \in X$ we have

$$\begin{aligned} h_{\text{perm}}(r \cdot z) &= \pi_{\text{perm}}(r \cdot z) \cdot (r \cdot z) = \pi_{\text{perm}}(z) \cdot (r \cdot z) \\ &= \pi_{\text{perm}}(z)\, r \pi_{\text{perm}}(z)^{-1}\, \pi_{\text{perm}}(z) \cdot z = \underbrace{\pi_{\text{perm}}(z)\, r \pi_{\text{perm}}(z)^{-1}}_{\in \mathbb{Z}_5^5} \cdot h_{\text{perm}}(z)\,. \end{aligned}$$

Notice that $\pi_{\text{perm}}(r \cdot z) = \pi_{\text{perm}}(z)$ because π_{perm} orders the entries in the z-tuple according to their norms and r acting on z only changes their arguments.

Performing a similar analysis, we find that $h_{\text{RootSc}} \circ h_{\text{perm}} \circ h_{\text{hom}}$ is a $[(\mathbb{C}^\star \times S_5) \ltimes \mathbb{Z}_5^5]$-canonicalization, where h_{hom} is given by (16), and Spectral $\circ\, h_{\text{perm}}$ is invariant under $S_5 \times \mathbb{C}^\star$. Further relevant facts about h_{hom} is that it commutes with h_{perm} but it does not commute with h_{RootSc}. Moreover, h_{hom} is irrelevant in combination with the spectral layer. We take this into account when we combine different layers in the experiments below.

5. Experiments

We have performed a number of experiments, on different CICY geometries, to compare the accuracy and performance of invariant ϕ-models, obtained through the canonicalization procedure described in section 4, with the standard and spectral ϕ-models. In this section, we present the results of these experiments. We observe an increase in accuracy, as measured by the loss components and error measures, in all examples. Moreover, in the case of a highly symmetry space, i.e. the Fermat quintic, we observe significantly faster convergence of the model upon training. We observe faster convergence of the Calabi-Yau volume prediction for canonicalized variants of the standard ϕ models. For spectral ϕ models, the opposite effect is observed.

We use the pointgenerator classes of the `cymetric` package to generate samples of 100 000 points on the CY spaces[19]. The data sets are split (1:9) into validation and training data sets. The experiments consist of runs over five different point sets, and we report the average and the standard deviation of these runs. We test the models on an unseen test set of 50 000 points.

In more detail, we set up the standard, dense ϕ-model with 3 hidden layers with 64 nodes each. The network is fully connected, has GELU activation function, Adam optimizer with 0.001 learning rate, and the weights of the custom losses of the `cymetric` package take default values. We use the default (`glorot_uniform`) initialization of `TensorFlow` for the trainable weights of the neural networks. These hyperparameters were chosen after an initial optimization study on the standard ϕ-model, and were kept constant across the invariant networks. The rationale for this choice was to isolate the effects of the invariant layers from effects of hyperparameter tuning. We leave a more extensive optimization study of the hyperparameters of invariant architectures for the future.

5.1. Fermat quintic

Working on the Fermat quintic, we compare the standard ϕ-model with networks where one, or several, non-trainable layers are added. The added non-trainable layers correspond to the spectral layer which, as described in section 3.3, maps the data to a form which is invariant under homogeneous rescalings. We also explore G-canonicalization layers that enforce invariance under homogeneous rescalings, permutations and scaling by roots of unity, as described in 4.1.1–4.1.3.

We have performed a number of experiments on this geometry, and report the results from training the models on 90 000 points, over 50 epochs in figures 3 and 4 (we have checked that the conclusions are reproduced in 300-epochs experiments). Since training and validation plots look very similar, we only report the latter[20]. Thus, figures 3 and 4 show the evolution, over the training epochs, of Monge-Ampère, Ricci and transition loss on a validation set consisting of 10 000 points (the Kähler loss evaluates to $\mathcal{O}(10^{-16})$, i.e. it evaluates to zero within numerical precision, and is omitted from the plot. Note, that this behaviour is expected, since this loss component should be zero by construction). We also track the evolution of the volume, as computed by the integral of the volume measure of the predicted metric (cf appendix B). Since the computation of the volume requires an integral, it is performed over the full data set, i.e. 100 0000 points.

As is clear from the MA loss (upper-left in figures 3 and 4), the spectral layer and canonicalization layers all have a positive impact on the loss. This is clearly visible at initialization of the network. After training, the

[19] For CY hypersurfaces, we use `PointGenerator`, for CICYs with product ambient space we use `CICYPointGenerator`.

[20] In more detail: the MA and transition losses on training data look very similar to the shown validation plots, showing that there is no problem with overfitting. The Kclass loss evolution is consistent with the Volume plot on validation data. Kähler and Ricci loss are disabled during training.

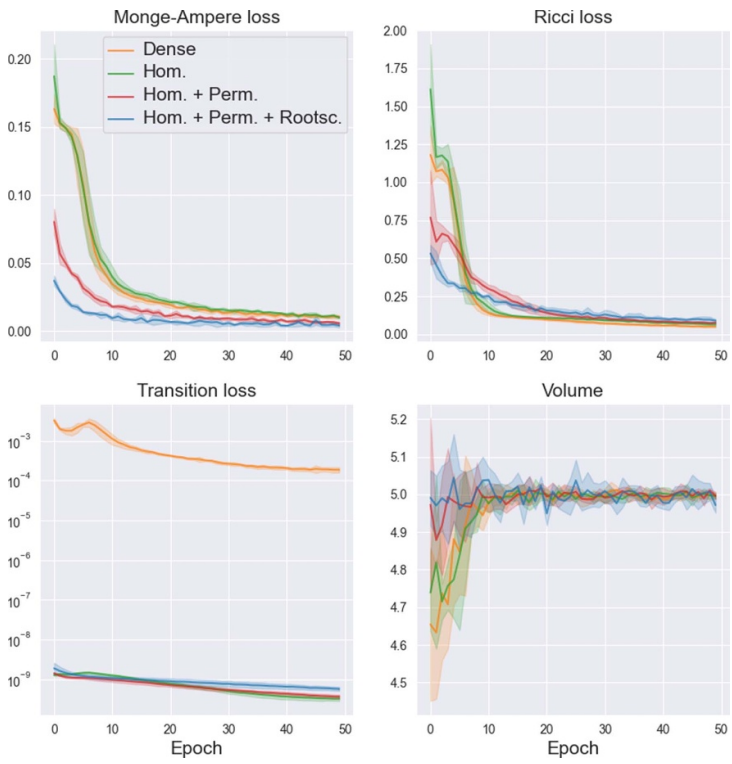

Figure 3. Performance of different ML models on the Fermat quintic. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 5). The different lines refer to dense networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

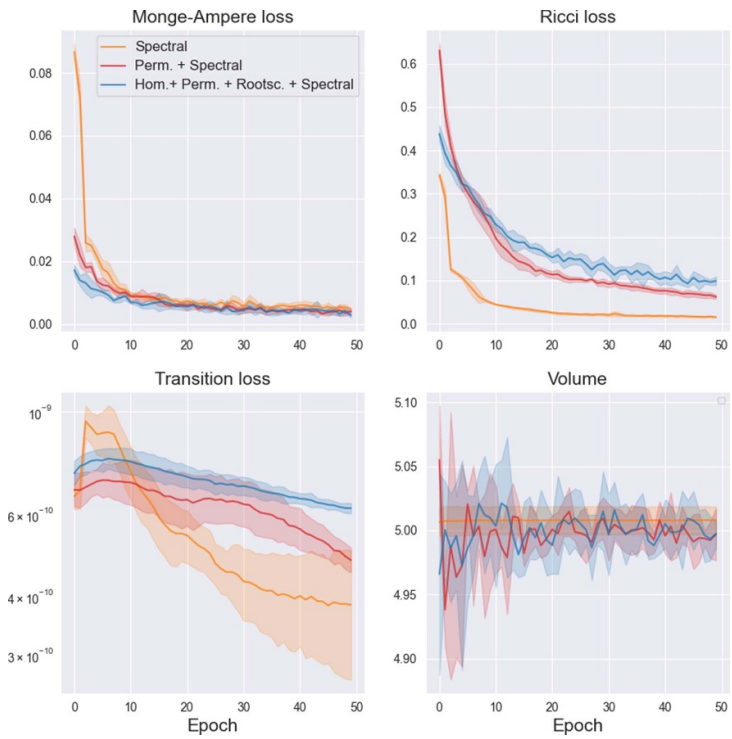

Figure 4. Performance of different ML models on the Fermat quintic. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 5). The different lines refer to spectral networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

Table 1. σ and $\mathcal{R}$ measure for the Fermat quintic evaluated over a test set of 50 000 points. The values are rounded to four decimals and the error is given by the standard deviation of five experiments.

Model	σ measure	$\mathcal{R}$ measure
FS metric	0.3720	0.6919
Dense	0.0106 ± 0.0011	0.0502 ± 0.0040
Hom.	0.0100 ± 0.0006	0.0645 ± 0.0052
Hom. + Perm.	0.0058 ± 0.0007	0.0754 ± 0.0066
Hom. + Perm. + Rootsc.	0.0040 ± 0.0012	0.0891 ± 0.0084
Spectral	0.0047 ± 0.0004	0.0149 ± 0.0011
Perm. + Spectral	0.0040 ± 0.0004	0.0600 ± 0.0049
Hom. + Perm. + Rootsc. + Spectral	0.0027 ± 0.0005	0.0956 ± 0.0091

effect of the invariant layers on the MA loss is still positive, albeit less pronounced. This is in accord with previous observations in the literature indicating that the standard ϕ-model is fairly good at learning symmetries.

When examining the Ricci loss, we notice a number of interesting features. First, the spectral layer has a larger positive impact on performance, both at initialization and during training. Second, while the canonicalization layers may have a positive impact on the Ricci loss at initialization of standard ϕ-models, the effect becomes negative after 10 epochs of training. While this discrepancy decreases with training, the standard ϕ-model has a slightly lower Ricci loss on validation than its canonicalized counterparts. For spectral models, adding canonicalized layers has a negative effect on the Ricci loss. We interpret this as an effect of a complicated loss landscape where optimizing with respect to the MA loss does not necessarily improve the Ricci loss (despite the fact that both of these losses are zero for the true Ricci-flat metric). Similar effects have been observed when learning CY metrics on more complicated CICY manifolds [40], and may here be related to the fact that the symmetry group has fixed points, that may affect the computation of Ricci loss. As suggested in reference [40], by making a different hyperparameter choice, so that both MA and Ricci losses are used to guide the training of the ML model, performance may be improved; we leave such studies for the future.

The lower-left subplots in figures 3 and 4 show the transition loss: this is $\mathcal{O}(10^{-3})$ for the dense network, and 6 orders of magnitude lower for the spectral network, and any network that includes a homogeneous canonicalization layer. This is in accordance with expectations, and indicates that $\mathbb{C}^*$-invariance is achieved.

Finally, the volumes computed with the trained models all converge to the ground truth upon training (within numerical errors). The spectral layer gives a constant volume under training, which shows that this model indeed predicts a globally defined ϕ at any time during training. For the other architectures, there is more variance, so the same claim cannot be made. The standard, dense ϕ-model exhibits most variation, and imposing G-canonicalization has a positive effect on the constancy of the volume.

In table 1, we list the σ and $\mathcal{R}$ measures, which correspond to the integrated MA and Ricci loss respectively (see appendix C), for the fully trained models as well as the measures for the Fubini-Study metric as a benchmark. We can see that adding the canonicalization layers has a positive effect on the σ measure. Concatenating the homogeneous, permutation and root scaling canonicalization layers improves the σ measure by two orders of magnitude compared to the Fubini-Study metric, and by a factor of 2 compared to the standard, dense ϕ-model. Similar improvement is observed for the spectral layer. For comparison, the σ measure of the best performing experiments in table 1 is 0.0027, which would correspond to $k \approx 33$ with Donaldson's algorithm, and $k \approx 5$ with Headrick-Nassar's functional minimization[21]. On the $\mathcal{R}$ measure, the spectral network performs the best. Any addition of canonicalization layers worsens the accuracy of the prediction, in particular when combined with the spectral layer. This is in accordance with the observed Ricci loss values on validation data, as discussed above.

5.2. Tetraquadric

In this example, we study the tetraquadric defined by the configuration matrix

$$\left[\begin{array}{c|c} 1 & 2 \\ 1 & 2 \\ 1 & 2 \\ 1 & 2 \end{array}\right]. \tag{24}$$

[21] This comparison uses the observations of σ measures for $k \in [1, 12]$, which shows that the σ measure decreases polynomially, like $\sigma \sim 3.1k^{-2} - 4.2k^{-3}$, for Donaldson's balanced metric [7], and exponentially, like $\sigma \sim 0.17 \times 2.2^{-k}$, for Headrick-Nassar's optimal metric [10].

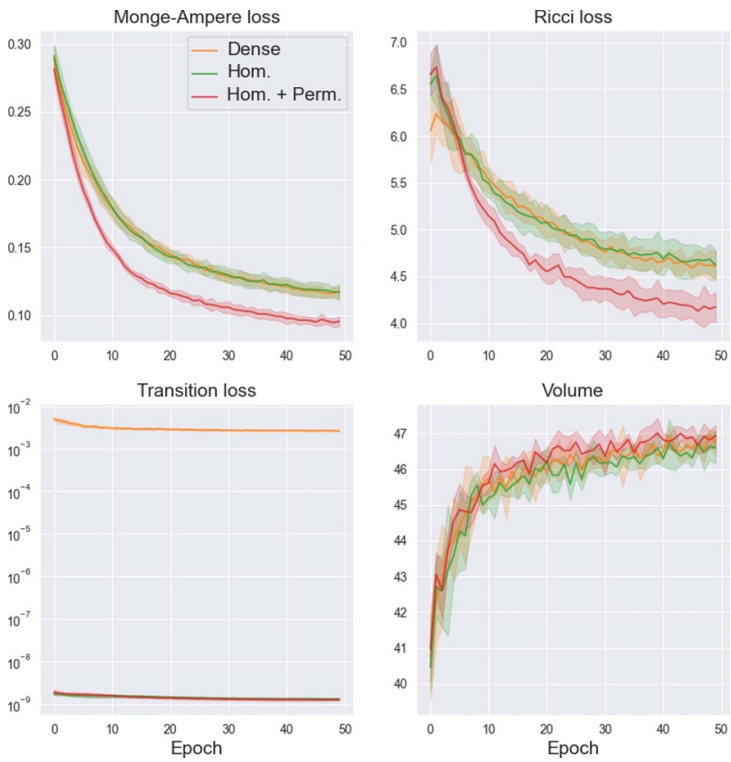

Figure 5. Performance of different ML models on a S_2 invariant tetraquadric CY 3-fold. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 48). The different lines refer to dense networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

This CY 3-fold has $h^{(1,1)} = 4$ and $h^{(2,1)} = 68$. We pick a smooth member of this CICY family, defined by a degree (2,2,2,2)-polynomial p of $\mathcal{M}$ which is invariant under the action of S_2, as defined in § 4.1.2. Concretely, we set

$$
\begin{aligned}
&p(x_0,x_1,y_0,y_1,z_0,z_1,w_0,w_1)\\
&\quad := 2x_0^2y_0^2z_0^2w_0^2 - 2x_0^2y_0^2z_0^2w_0w_1 + 2x_0x_1y_0^2z_0^2w_1^2 + x_1^2y_0^2z_0^2w_0^2 + 2x_1^2y_0^2z_0^2w_0w_1\\
&\qquad + x_0^2y_0y_1z_0^2w_0^2 + x_0^2y_0y_1z_0^2w_1^2 - 2x_0x_1y_0y_1z_0^2w_0w_1 - x_0x_1y_0y_1z_0^2w_1^2\\
&\qquad + x_1^2y_0y_1z_0^2w_0^2 - x_1^2y_0y_1z_0^2w_1^2 - 2x_0^2y_1^2z_0^2w_0^2 - x_0^2y_1^2z_0^2w_0w_1 - 2x_0^2y_1^2z_0^2w_1^2\\
&\qquad - 2x_0x_1y_1^2z_0^2w_0^2 - 2x_0x_1y_1^2z_0^2w_0w_1 + x_1^2y_1^2z_0^2w_0^2 + 2x_1^2y_1^2z_0^2w_0w_1 + 2x_1^2y_1^2z_0^2w_1^2\\
&\qquad + x_0^2y_0^2z_0z_1w_0^2 - 2x_0^2y_0^2z_0z_1w_1^2 - x_0x_1y_0^2z_0z_1w_0^2 - 2x_0x_1y_0^2z_0z_1w_0w_1\\
&\qquad + x_0x_1y_0^2z_0z_1w_1^2 - x_1^2y_0^2z_0z_1w_0^2 + x_1^2y_0^2z_0z_1w_0w_1 - 2x_1^2y_0^2z_0z_1w_1^2\\
&\qquad - x_0^2y_0y_1z_0z_1w_0w_1 + 2x_0^2y_0y_1z_0z_1w_1^2 - x_0x_1y_0y_1z_0z_1w_0^2 - 2x_0x_1y_0y_1z_0z_1w_0w_1\\
&\qquad - x_0x_1y_0y_1z_0z_1w_1^2 + 2x_1^2y_0y_1z_0z_1w_0^2 - x_1^2y_0y_1z_0z_1w_0w_1 - 2x_0^2y_1^2z_0z_1w_0^2\\
&\qquad + x_0^2y_1^2z_0z_1w_0w_1 - x_0^2y_1^2z_0z_1w_1^2 + x_0x_1y_1^2z_0z_1w_0^2 - 2x_0x_1y_1^2z_0z_1w_0w_1\\
&\qquad - x_0x_1y_1^2z_0z_1w_1^2 - 2x_1^2y_1^2z_0z_1w_0^2 + x_1^2y_1^2z_0z_1w_1^2 + 2x_0^2y_0^2z_1^2w_0^2 + 2x_0^2y_0^2z_1^2w_0w_1\\
&\qquad + x_0^2y_0^2z_1^2w_1^2 - 2x_0x_1y_0^2z_1^2w_0w_1 - 2x_0x_1y_0^2z_1^2w_1^2 - 2x_1^2y_0^2z_1^2w_0^2\\
&\qquad - x_1^2y_0^2z_1^2w_0w_1 - 2x_1^2y_0^2z_1^2w_1^2 - x_0^2y_0y_1z_1^2w_0^2 + x_0^2y_0y_1z_1^2w_1^2\\
&\qquad - x_0x_1y_0y_1z_1^2w_0^2 - 2x_0x_1y_0y_1z_1^2w_0w_1 + x_1^2y_0y_1z_1^2w_0^2 + x_1^2y_0y_1z_1^2w_1^2\\
&\qquad + 2x_0^2y_1^2z_1^2w_0w_1 + x_0^2y_1^2z_1^2w_1^2 + 2x_0x_1y_1^2z_1^2w_0^2 - 2x_1^2y_1^2z_1^2w_0w_1 + 2x_1^2y_1^2z_1^2w_1^2\,.
\end{aligned}
$$

Note that this instance of the tetraquadric has no root scaling invariance. We project to the fundamental domain associated to S_2 as defined in equation (21). For the homogenous rescalings, we consider each ambient space sperately and project into the fundamental domain as in equation (16). The spectral layer is realized as described in section 3.3 for a product of ambient spaces.

We perform analogous experiments as in the previous section, and report the validation results for five experiments trained over 90 000 points for 50 epochs in figures 5 and 6. As expected from the previous experiment, adding the homogeneous canonicalization layer leads to an almost vanishing transition loss

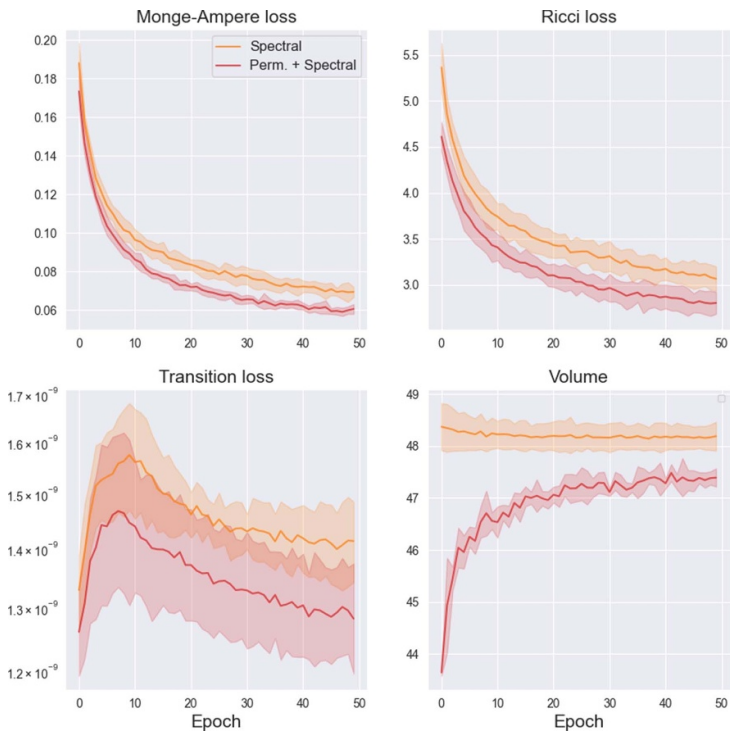

Figure 6. Performance of different ML models on a S_2 invariant tetraquadric CY 3-fold. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 48). The different lines refer to spectral networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

Table 2. σ and $\mathcal{R}$ measure for the tetraquadric evaluated over a test set of 50 000 points. The values are rounded to four decimals and the error is given by the standard deviation of five experiments.

Model	σ measure	$\mathcal{R}$ measure
FS metric	0.4393	7.0902
Dense	0.1168 ± 0.0020	4.5680 ± 0.0823
Hom.	0.1183 ± 0.0017	4.5906 ± 0.0481
Hom. + Perm.	0.0960 ± 0.0018	4.1339 ± 0.0882
Spectral	0.0665 ± 0.0002	2.9522 ± 0.0373
Perm. + Spectral	0.0582 ± 0.0007	2.6980 ± 0.0326

while it does not improve the performance on the Monge–Ampère and Ricci loss. However, adding the permutation canonicalization layer to the architecture improves performance both on Monge–Ampère and Ricci loss. The spectral layer performs slightly better than the previous configuration, while the combination of permutation canonicalization and spectral layer performs best.

For the volume, we make the following observations. The spectral layer is most stable over all epochs and follows, within a numerical error, the ground truth of 48 while all other combinations show higher variance and seem to converge to a slightly shifted value. As discussed in section 4, the variance could be explained due to the fact that the fundamental domain projections introduce discontinuities which might skew calculations, like the volume, which require integration over the entire manifold. However, the shifted volume prediction on the tetraquadric is not observed in the other experiments we have performed. The explanation behind this difference is likely that $h^{(1,1)} > 1$ for the tetraquadric, while it is 1 for the other CY manifolds, and that using a loss function to fix the Kähler class during training is more challenging on manifolds with higher $h^{(1,1)}$.

In table 2, we compute the σ and $\mathcal{R}$ measures for the different models. For the σ measure, we observe a similar trend as for the Fermat quintic. Namely, adding any combination of canonicalization layers[22] has a positive effect on the measure. The spectral layer shows the most significant improvement, while the combination of permutation canonicalization layer and spectral layer perform the best. The $\mathcal{R}$ measure shows a different trend as for the Fermat quintic. Adding the permutation canonicalization layer improves

[22] Adding only the homogeneous canicalization layer does not improve the σ measure. The main purpose of this layer is to reduce the transition loss.

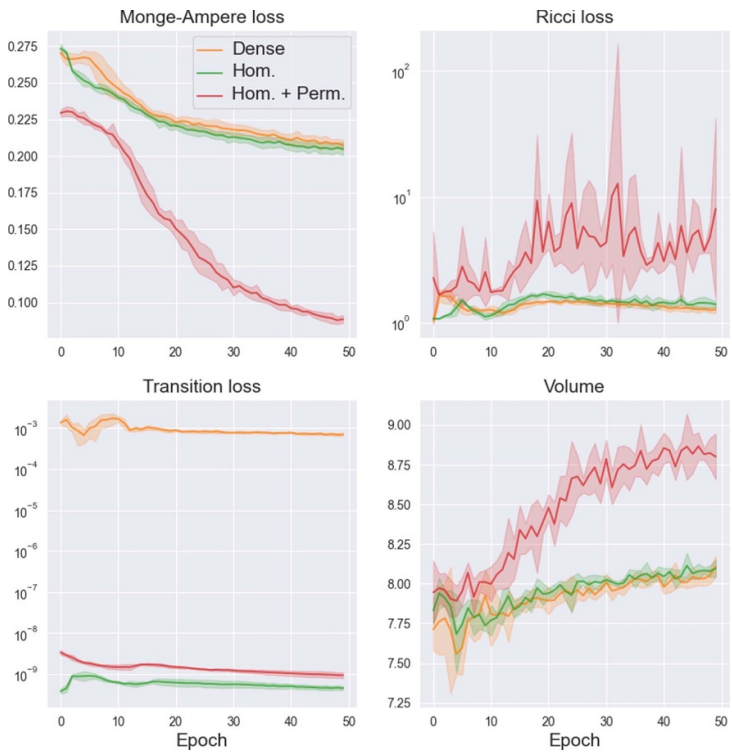

Figure 7. Performance of different ML models on a [5|3 3] CY 3-fold. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 9). The different lines refer to dense networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

the measure slightly, while again the spectral layer shows the most significant improvement. Again, the combination of permutation canonicalization layer and spectral layer performs the best.

5.3. A 2-polynomial CICY

We look at the CICY [5|3 3], where the two defining polynomials are given by (cf [32])

$$p(x_0,x_1,x_2,x_3,x_4,x_5) := x_0^3 + x_1^3 + x_2^3 - 3x_3x_4x_5,$$
$$q(x_0,x_1,x_2,x_3,x_4,x_5) := x_3^3 + x_4^3 + x_5^3 - 3x_0x_1x_2.$$

This CICY has a discrete group $\mathbb{Z}_2(S_3 \times S_3)$ of permutation symmetries, where the non-trivial element in $\mathbb{Z}_2$ acts by sending $[x_0 : x_1 : x_2 : x_3 : x_4 : x_5]$ to $[x_3 : x_4 : x_5 : x_0 : x_1 : x_2]$. while the first S_3 factor permutes the coordinates (x_0, x_1, x_2) and the second S_3 factor permutes the coordinates (x_3, x_4, x_5). The group does not act freely.

We implement a canonicalization for the action of $\mathbb{Z}_2(S_3 \times S_3)$ in two steps. In the first step, we reorder the coordinates in the first triple and the second triple in descending order with respect to norm, separately

$$[x_0 : x_1 : x_2 : x_3 : x_4 : x_5] \mapsto [x_{\sigma(0)} : x_{\sigma(1)} : x_{\sigma(2)} : x_{\tau(3)} : x_{\tau(4)} : x_{\tau(5)}]$$
$$\text{where } (\sigma,\tau) \in S_3 \times S_3,\ |x_{\sigma(0)}| \geqslant |x_{\sigma(1)}| \geqslant |x_{\sigma(2)}|,\ |x_{\tau(3)}| \geqslant |x_{\tau(4)}| \geqslant |x_{\tau(5)}|.$$

In the second step, we shift such that the triple with highest sum of norms is first. We combine these two steps in one permutation canonicalization layer.

We perform experiments similar to the previous sections where we combine homogeneous and permutation canonicalization layers, and add them to the dense and spectral ϕ-models. In figures 7 and 8, we see that the homogeneous canonicalization layer leads to an almost vanishing transition loss when combined with the dense network. Regarding volume computation, we can see that adding the permutation canonicalization layer helps the dense network to converge faster to the correct volume, but it worsens the computation for the spectral network. With respect to the Monge–Ampère loss, the permutation canonicalization layer improves the performance of both networks by a factor of 2. In contrast to the previous experiments, the Ricci loss shows different trends from the Monge–Ampère loss. Only the spectral network reaches a clear plateau value after 10 epochs, while the other networks seems to oscillate. The

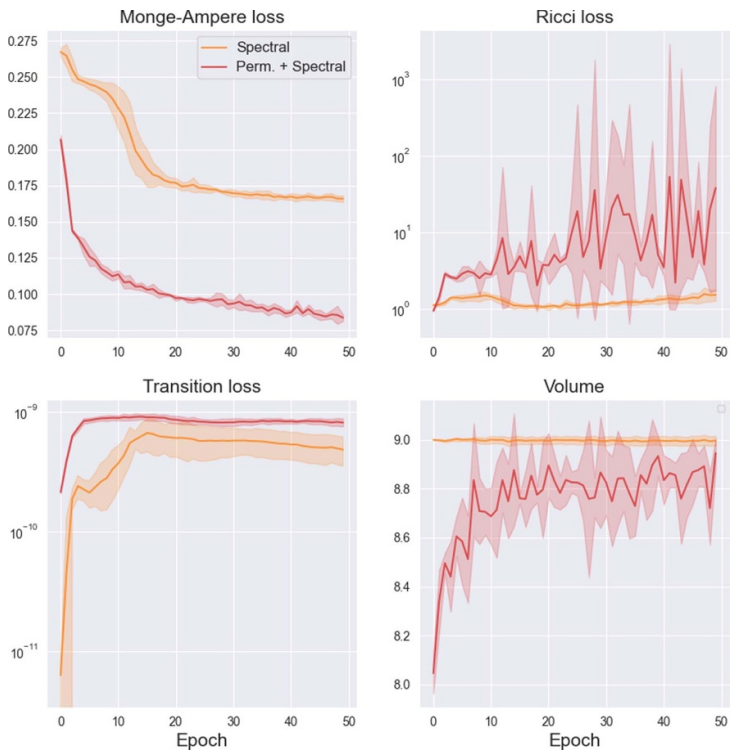

Figure 8. Performance of different ML models on a [5|3 3] CY 3-fold. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 9). The different lines refer to spectral networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

Table 3. σ and $\mathcal{R}$ measure for the [5|3 3] CY 3-fold evaluated over a test set of 50 000 points. The values are rounded to four decimals and the error is given by the standard deviation of five experiments.

Model	σ measure	$\mathcal{R}$ measure
FS metric	0.2620	1.0907
Dense	0.1971 ± 0.0015	0.9796 ± 0.1174
Hom.	0.1974 ± 0.0020	1.0003 ± 0.0463
Hom. + Perm.	0.1983 ± 0.0006	1.2831 ± 0.1380
Spectral	0.1944 ± 0.0002	0.7203 ± 0.0291
Perm. + Spectral	0.1981 ± 0.0007	2.2978 ± 2.5123

permutation canonicalized models has an initial rise followed by a steep descent; since the model has not settled, further training might improve the Ricci loss, and training against the Ricci loss, in addition to the MA loss, might also improve performance, as discussed for the Fermat quintic experiments.

In table 3, we compute the σ and $\mathcal{R}$ measures for the different models. For the σ measure we observe almost identical performance for all architectures. The $\mathcal{R}$ measure does not improve for any architecture except the spectral model, which is in accordance with the Ricci loss validation plots. The explanation is likely that the loss landscape for this geometry is hard to navigate, and that models get stuck in a local minimum. This is supported by the observation that even the standard, dense ϕ-model shows poorer performance on the $\mathcal{R}$ measure than on the other geometries.

6. Ricci-flat metrics on non simply connected CY manifolds

In heterotic string compactifications, phenomenologically interesting particle physics models are often based on CY manifolds that are not simply connected. Such manifolds admit Wilson lines, which allow us to break the heterotic E_8 group to the gauge group of the Standard Model, and achieve the correct number of particle generations; this idea is pedagogically explained in [56], and dates back to [57–59]. There is a large literature on the use of discrete symmetries in heterotic model building, and the reader is referred to the papers cited above for more discussion on this topic. In this section, we will follow [8] and study the Ricci-flat metric on a quotient Q of a smooth, 5-parameter quintic X by a freely acting $\mathbb{Z}_5^2$ group.As explained e.g. in [8, 56], Q has

Euler characteristic -8, and leads to a particle physics model with 4 generations, so we will refer to this geometry as a 4-generation quotient of the quintic.

In more detail, we will study the 5-parameter quintic CY specified, in $\mathbb{CP}^4$, by the zero locus of

$$\begin{aligned} p = &\left(z_0^5 + z_1^5 + z_2^5 + z_3^5 + z_4^5\right) \\ &+ \psi_1 z_0 z_1 z_2 z_3 z_4 \\ &+ \psi_2 \left(z_0^3 z_1 z_4 + z_0 z_1^3 z_2 + z_0 z_3 z_4^3 + z_1 z_2^3 z_3 + z_2 z_3^3 z_4\right) \\ &+ \psi_3 \left(z_0^2 z_1 z_2^2 + z_1^2 z_2 z_3^2 + z_2^2 z_3 z_4^2 + z_3^2 z_4 z_0^2 + z_4^2 z_0 z_1^2\right) \\ &+ \psi_4 \left(z_0^2 z_1^2 z_2 + z_1^2 z_2^2 z_3 + z_2^2 z_3^2 z_4 + z_3^2 z_4^2 z_0 + z_4^2 z_0^2 z_1\right) \\ &+ \psi_5 \left(z_0^3 z_2 z_3 + z_1^3 z_3 z_4 + z_2^3 z_4 z_0 + z_3^3 z_0 z_1 + z_4^3 z_1 z_2\right) . \end{aligned} \tag{25}$$

We pick a generic instance of this family of CY manifolds by drawing ψ_i from $\mathcal{N}(0,1)$.

As is shown in reference [8], this 5-parameter quintic X is invariant under a group $G = \mathbb{Z}_5^2$ generated by

$$g_1 = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} , \; g_2 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & e^{\frac{2\pi i}{5}} & 0 & 0 & 0 \\ 0 & 0 & e^{\frac{4\pi i}{5}} & 0 & 0 \\ 0 & 0 & 0 & e^{\frac{6\pi i}{5}} & 0 \\ 0 & 0 & 0 & 0 & e^{\frac{8\pi i}{5}} \end{pmatrix} . \tag{26}$$

Moreover, the group G is shown to act freely on X, so that the quotient $Q = X/G$ is smooth.

In [8], Braun and collaborators obtain an approximation for the Ricci-flat CY metric on Q using the Donaldson algorithm. As was mentioned in the introduction of this paper, this algorithm computes the CY metric via an expansion in a spectral basis, consisting of degree-k homogeneous polynomials, using a fixed point iteration scheme to determine the coefficients of the expansion. The accuracy of metric improves as $k \to \infty$. The discrete symmetry is imposed as a restriction on the spectral basis, and results in a slower growth of the basis dimension. The invariant spectral basis can thus be constructed for larger k than the non-invariant one; consequently more accurate approximations for the metric can be achieved.

However, it is somewhat non-trivial to determine the basis of sections on Q, i.e. identify the linearly independent G-invariant sections on X that restrict to Q, already for moderate values of k. There are two subtleties: first, polynomials of the form $q_i p$, where $p = 0$ defines the CY hypersurface, evaluate to zero on the CY, and must be removed from the invariant basis. Second, the generators g_1, g_2 only commute up to homogeneous rescaling of the coordinates, which makes the construction of invariant polynomials non-trivial. Braun and collaborators rely on algorithms of invariant theory, implemented using SINGULAR [60], to identify the basis, and explore these methods up to $k = 25$. After running the Donaldson algorithm on a a sample with 10^6 points, they find that the σ measure evaluates to 0.015 for $k = 25$ on an unseen test set of 20 000 points[23].

In this section, we revisit the computation of the Ricci-flat CY metric on Q using the methods introduced in section 4. We thus encode the G-invariance as non-trainable canonicalization layers in the ϕ-model of `cymetric` , and compare the performance of these invariant ML models with the standard dense counterpart, as well as spectral networks. Notice that the predicted metric using the above canonicalization layers is on the covering space X, rather than being on the quotient Q: the metric is invariant under the action of G, and so constant along the fibers, but it is expressed in coordinate patches on X. Subsequently, in order to obtain the metric on the quotient, one needs to pushforward the learned metric via the projection from X to Q. In this work, we do not execute this last step.

We implement the canonicalization layers as follows. Let $G_i \subset G$ be the subgroup generated by the generator g_i for $i = 1, 2$. Then, by projecting on the fundamental domains of G_1 and G_2 consecutively, we construct an invariant function on X w.r.t G or, in other words, a function on the smooth quotient $Q = X/G$. A technical complication, compared to the Fermat quintic canonicalization layers of section 5.1, is that we need to address the non-commutativity of G_1 and G_2. This can be resolved by inserting the homogeneous canonicalization layer before the other G-canonicalization layers; g_1 and g_2 commute up to homogeneous rescalings.

The fundamental domains of G_1 and G_2 are given by

$$\mathcal{F}_1 := \left\{ [z_0 : z_1 : z_2 : z_3 : z_4] \in \mathbb{CP}^4 | \, \forall i : |z_0| \geqslant |z_i| \right\} , \tag{27}$$

[23] When comparing the performance against number of basis elements, they find that the G-invariant metric approximation has smaller σ error than a metric computed using the unrestricted spectral basis.

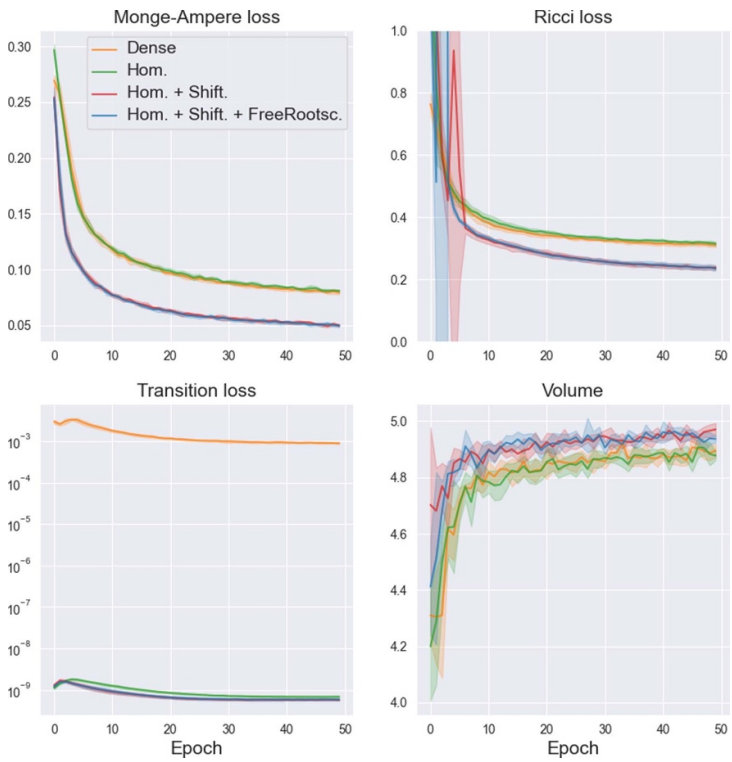

Figure 9. Performance of different ML models on a 4-generation quintic. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 5). The different lines refer to dense networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

$$\mathcal{F}_2 := \left\{ [z_0 : z_1 : z_2 : z_3 : z_4] \in \mathbb{CP}^4 \middle| \, 0 \leqslant \arg(z_0) \leqslant 2\pi/5 \right\}. \tag{28}$$

To project on the fundamental domains $\mathcal{F}_1$ and $\mathcal{F}_2$, we implement two canonicalization layers that act on the homogeneous coordinates $[z_0 : z_1 : z_2 : z_3 : z_4]$ of a point on X. The first layer returns the coordinate tuple shifted to the left such that the highest norm entry takes the first spot,

$$h_{\text{shift}} : (z_0, z_1, z_2, z_3, z_4) \mapsto \left(z_i, z_{(i+1 \mod 5)}, z_{(i+2 \mod 5)}, z_{(i+3 \mod 5)}, z_{(i+4 \mod 5)}\right), \quad \forall j : \; |z_i| \geqslant |z_j|.$$

For the second canonicalization layer, we find the fifth root of unity a such that $0 \leqslant \arg(az_0) \leqslant 2\pi/5$ and then multiply the rest of the coordinates with successive powers of a as follows,

$$h_{\text{FreeRootSc}} : (z_0, z_1, z_2, z_3, z_4) \mapsto \left(az_0, a^2 z_1, a^3 z_2, a^4 z_3, z_4\right).$$

Our experiments run over 50 epochs with the same settings as in section 5. The results on validation data are shown in figures 9 and 10 (we again omit the training plots which are more or less identical to the validation plots). Just as observed in other experiments, including a homogeneous canonicalization layer, or a spectral layer, significantly reduces the transition loss. The validation plots of Monge-Ampère and Ricci loss shows that canonicalization with respect to G_1, G_2 reduces the losses, albeit not by a large factor[24]. The reduction of the loss is more pronounced when G-canonicalization is added to the dense model, than when similar additions are made in the spectral network. The addition of canonicalization layers also has a small positive effect on the volume for the dense network. For the spectral network, the effect on the volume prediction is instead negative at initialization. While this effect is subdued by training, it is an indication that the canonicalization layers introduce discontinuities, which affect global predictions of the model in a negative way.

In table 4, we compute the σ and $\mathcal{R}$ measure for all the models. We can observe an improvement on the σ measure by adding any combination of canonicalization layers to the standard, dense ϕ-model[25]. The

[24] The spikes in the Ricci loss in the upper-right plot in figure 9 are transient numerical fluctuations.

[25] Again, except for the case in which only the homogeneous canonicalization layer is added, which performs on par with the dense network.

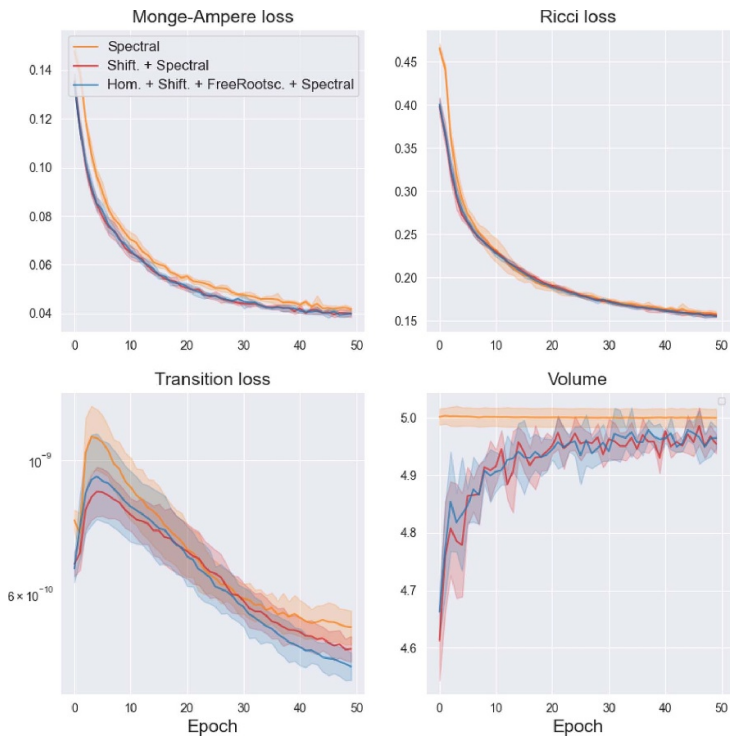

Figure 10. Performance of different ML models on a 4-generation quintic. The first row shows Monge–Ampère and Ricci loss on validation data. The second row shows transition loss evolution on validation data; and the volume computed with trained metric on all data (ground truth is 5). The different lines refer to spectral networks with and without G-canonicalizations. Solid line is average over 5 experiments, lighter color bands show the standard deviation.

Table 4. σ and $\mathcal{R}$ measure for the 4-generation quotient of the quintic evaluated over a test set of 50 000 points. The values are rounded to four decimals and the error is given by the standard deviation of five experiments.

Model	σ measure	$\mathcal{R}$ measure
FS metric	0.4006	0.7876
Dense	0.0802 ± 0.0007	0.3070 ± 0.0048
Hom.	0.0818 ± 0.0011	0.3115 ± 0.0053
Hom. + Shift	0.0512 ± 0.0009	0.2374 ± 0.0039
Hom. + Shift + FreeRootsc.	0.0523 ± 0.0012	0.2396 ± 0.0062
Spectral	0.0419 ± 0.0004	0.1571 ± 0.0026
Shift + Spectral	0.0404 ± 0.0014	0.1570 ± 0.0032
Hom. + Shift + FreeRootsc. + Spectral	0.0399 ± 0.0011	0.1549 ± 0.0017

spectral network performs better than any model based on the standard model, and combining the spectral layer with canonicalization layers gives the lowest σ measure. For comparison, the error measure values obtained by the fully canonicalized dense (spectral) networks correspond to the value obtained with k about 15 (20) using Donaldson's algorithm [8]. For the $\mathcal{R}$ measure, we observe the same trend as for the σ measure.

Summing up the results of this experiment, we note that the canonicalization approach is easily implemented on this quintic, and that the layers combine nicely with both the dense and spectral ϕ-models. Combining this with the pushforward, via the projection from X to Q, we can thus learn the metric on quotient CY manifolds using these machine learning techniques. Furthermore, the accuracy of the prediction, as measured by the σ and $\mathcal{R}$ measures, improves, and compares well to the results of reference [8]. We leave a more rigorous study of the performance gains that can be attained with these models, compared to previous work, for future hyperparameter optimization studies. What we can conclude already now is that, since there is no need to construct the G-invariant spectral basis, the G-canonicalization layers seem to provide a technical simplification compared to spectral methods used in reference [8]

7. Conclusion

In this paper, we have addressed the question: how do we encode symmetries in a machine learned Ricci-flat CY metric? From a geometric point of view, CY manifolds may have discrete symmetries, with interesting

consequences in mathematics and physics. From a machine learning perspective, these symmetries are manifest as invariances, under group transformations, of the machine learning models designed to predict CY metrics. By constructing invariant ML models, one expects better performance and accuracy. A clear application of these invariant models are in string phenomenology, for example the computation of heterotic Yukawa couplings, which was recently explored using machine learning tools in [32, 33].

Our work is inspired by recent developments in the ML literature, that advocate to achieve model invariance by a pre-processing step, where the input data is projected into a fundamental domain of the relevant group [35, 36]. We have used an approach where non-trainable layers perform these projections; collectively, we refer to these layers as canonicalization layers. This is a modular implementation, and the layers can easily be exchanged to match different settings. While we have limited our studies to the `cymetric` package [21, 22], we expect similar canonicalization layers can be added with ease in other ML packages for CY geometry.

We have performed experiments on different CICY manifolds, comparing the performance of the ϕ-model of the `cymetric` package, which is considered the state of the art model for approximating Calabi-Yau metrics with machine learning, with and without canonicalization layers, but otherwise identical hyperparameter settings. We have also compared with the spectral network, originally proposed by Berglund and collaborators [24], which may also be combined with canonicalization layers. We find that adding canonicalization layers increases performance on the losses that are used to train the networks. In particular, the increase in performance on the transition loss is several orders of magnitudes for all architectures. The volume, controlled by the K-class loss, converges faster if canonicalization layers are added to the standard ϕ-model. However, the volume seems to be captured best by the spectral networks without canonicalization. Being a global property, this could be explained by discontinuities, related to the boundaries of fundamental domains, that the canonicalization layers may introduce. These are not present in the spectral layer.

The improvement on the Monge–Ampère loss is noticeable but not very large. Combining the spectral layer with the canonicalization layers seems to additionally increase performance while it introduces variance in the global prediction of the volume. However, adding these custom layers can still be of use regarding performance, as in many cases the losses converge faster, allowing one to reduce the number of training epochs. In addition, it has not yet been tested how the canonicalized layers perform in a systematic hyperparameter optimization study.

The Ricci loss yields different results depending on which geometry we look at. For the tetraquartic, as well as the 4-generation quotient of the quintic, we are quotienting by a freely acting symmetry group and we observe that canonicalization improves the Ricci loss on validation data, under training governed by the MA loss. The $\mathcal{R}$ error measures also decrease for canonicalized networks, compared to the standard dense and spectral ϕ-models. For the Fermat quintic and the $[5|3\ 3]$ CY 3-fold, adding the canonicalization layers instead increases the Ricci loss on validation data, and leads to a higher Ricci measure on test data. This could be explained by a more complicated loss landscape on these geometries, where training against the MA loss does not directly decrease the Ricci loss. The fact that the canonicalization layers encode a symmetry group that does not act freely on these geometries may contribute to this complicated loss landscape.

Furthermore, we would like to emphasize the advantage of the canonicalization layers following the philosophy of the Geometric Deep Learning program. The unrestricted models, consisting solely of dense layers, seem to already learn the symmetry of the data. This is in accord with previous observations in the literature, see e.g. [27], showing that the performance of the same dense network improves slightly on symmetric CY manifolds, compared to generic ones. However, adding the canonicalization layers creates an architecture that knows about the symmetries *a priori*, allowing for better interpretability of the model. Namely, we restrict the function space in which the model can learn to that of invariant functions, allowing us to better understand what the model actually predicts.

Moreover, using the canonicalization approach, we have taken steps towards learning metrics of non-simply connected CY manifolds. Such manifolds, Q, always have a simply connected CY manifold X as a covering space; the relation between the two spaces is $Q = X/\Gamma$, where Γ is a freely acting discrete symmetry group. The canonicalization idea provides a simple way of imposing Γ-invariance on the learned Ricci-flat metric of X, and thus, by pushforward, we can obtain the Ricci-flat metric of Q. While the approximation of CY quotient metrics is already well-studied, using other techniques [8, 10], the simplicity of the canonicalization layers, and the fact that they can be combined with relative ease, makes for a practical method, that can easily be adapted to different CY spaces and groups.

Looking ahead, there are many other directions yet to be explored. We have already mentioned that a thorough hyperparameter study might improve the performance of canonicalized architectures. Once this is done, it would be interesting to establish if scaling laws, as first observed in reference [61], hold for these ML models, and use these laws to compare the efficiency of different architectures. One could also explore the alternative invariant architectures suggested in reference [31], i.e. construct invariant layers using polynomial

invariants. While we have performed some investigations using this approach, they indicated that such layers quickly become too wide to be practically useful, even for the discrete symmetries prevalent among CY manifolds. In addition, this approach lacks the modular aspect of the canonicalization layers. A related avenue is to construct invariant models based on the invariant spectral basis to higher orders in k. This is similar to what has been attempted for algebraic CY metrics in the past, e.g. [7, 8, 10], so one would expect performance gains, even if we expect the construction of such invariant layers face similar computational subtleties as reported in [8].

Finally, there are many interesting phenomenological applications of G-invariant ML models, such as the numerical computation of Hermitian Yang Mills connections [62] and Yukawa couplings [32, 33]. We expect that the canonicalization approach will be particularly relevant for heterotic compactifications with non-standard embedding of the gauge group; as shown in the examples explored in [33], several neural nets must be trained to learn the Ricci-flat CY metric, Hermitian-Yang-Mills connection and harmonic representatives of certain vector bundle cohomologies. The G-invariance-imposing canonicalization layers can be applied to all of these networks; any gain in the computation cost achieved by the canonicalization layer will be multiplied by the number of networks to which it is applied. We expect this to be of importance for phenomenological studies.

Data availability statement

All data that support the findings of this study are included within the article (and any supplementary files).

Acknowledgments

This work was in part supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. Support was also obtained from Vetenskapsrådet, Grant no. 2020-03230, and from Uppsala University's AI4Research center. The computations were partly enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by Vetenskapsrådet through Grant agreement no. 2022-06725. ML thanks the Erwin Schrödinger Institute in Vienna, and the organisers of the program *The Landscape vs. the Swampland*, for hospitality during the completion of this work. MW thanks Nordita for hospitality during the completion of this work.

We thank Lara Anderson, Georgios Dimitroglou Rizell, Cristofero Fraser-Taliente, James Halverson, Thomas Harvey, Daniel Platt, Carlos Rodriguez, Fabian Ruehle, Robin Schneider, and Ayca Özcelikkale for valuable discussions and suggestions. We also thank Carlos Rodriguez and Robin Schneider for comments on a draft of this paper; Fabian Ruehle for his continuous work in maintaining the `cymetric` package; Muyang Liu and Martin Svärdsjö for collaborations on related topics; and, finally, Daniel Persson and Jan Gerken for organizing the Mathy-AI Journal Club, as well as its attendants, for providing a lot of inspiration for this work.

Appendix A. CY geometry

In this section, we prove that symmetries of the defining polynomial of a CICY manifold are isometries of the Ricci-flat metric. This should be well-known, but we have not found the proof in the standard references on the subject, which motivates including this section.

Note that the notation in this section differs from other sections, where J is used to define the Kähler form. Here, we will follow the prevalent notation of the mathematical literature and use ω for the Kähler form, and J for a complex structure.

We start with some preliminary definitions from section 8.4 in [63] . Let X be a complex manifold with a complex structure J. Then a hermitian metric g on X is a Riemann metric that preserves the complex structure i.e $g(J\cdot, J\cdot) = g(\cdot,\cdot)$. Given a hermitian metric g on X, we can define a 2-form $\omega(\cdot,\cdot) := g(J\cdot,\cdot)$ on X which is called the *Kähler* form, and X is called *Kähler* if its Kähler form is closed, i.e. $d\omega = 0$. Hence a Kähler manifold is equipped with three structure compatible structures J, g and ω. A fact from Kähler geometry tells us that given any two of the three structures, the compatibility conditions between them completely determine the third one. This immediately implies that if a given map between Kähler manifolds preserve any two of the structures then it preserves the third one. Finally, a map $f: X \to X$ is holomorphic if and only if it satisfies the Riemann-Cauchy equation given by

$$J \circ \mathrm{d}f = \mathrm{d}f \circ J.$$

Definition A.1. A Calabi-Yau manifold X is a compact Kähler manifold with a vanishing first Chern class, or equivalently with a nowhere-vanishing holomorphic top form.

Let X be a Calabi-Yau manifold embedded in a product of complex projective spaces. The Fubini-Study form restricts to a Kähler form ω_{FS} on X. The Calabi-Yau conjecture, or more precisely theorem, states that there exists a unique smooth function $\phi : X \to \mathbb{R}$, up to addition of a constant, such that the hermitian metric g_{CY} associated with the form $\omega_{CY} := \omega_{FS} + i\partial\bar{\partial}\phi$ is Ricci-flat. In the following propositions, we fix X to be a Calabi-Yau manifold embedded in a product of projective spaces.

Proposition 1. *Let $f : X \to X$ be a biholomorphic isometry of g_{FS} then f is an isometry of the Ricci-flat metric g_{CY} on X i.e $f^* g_{CY} = g_{CY}$.*

Proof. The map f preserves the Fubini-Study form ω_{FS} on X. Using this and the fact that the pullback via a holomorphic map commutes with the Dolbeault operators, we get

$$f^*\omega_{CY} = f^*\omega_{FS} + if^*\partial\bar{\partial}\phi = \omega_{FS} + i\partial\bar{\partial}(\phi \circ f)\,.$$

Notice that $f^* g_{CY}$ is the associated hermitian metric with $f^*\omega_{CY}$, and

$$Ric(f^* g_{CY}) = f^* Ric(g_{CY}) = 0\,.$$

Therefore, by the uniqueness part of the Calabi-Yau theorem, we have $f^*\omega_{CY} = \omega_{CY}$ and thus $f^* g_{CY} = g_{CY}$. □

Proposition 2. *Let $\phi : X \to \mathbb{R}$ be a smooth function that satisfies the equation $\omega_{CY} = \omega_{FS} + i\partial\bar{\partial}\phi$. Assume that $\mathcal{S}$ is a finite group of biholomorphic symmetries of ω_{FS}. Then ϕ is invariant under the action $\mathcal{S}$, i.e. for all $f \in \mathcal{S}$, we have $f^*\phi = \phi \circ f = \phi$.*

Proof. By proposition 1, we have $\mathcal{S}$ is a group of isometries of ω_{CY}. Consequently, for all $f \in \mathcal{S}$, we have

$$\omega_{FS} + i\partial\bar{\partial}\phi = \omega_{CY} = \omega_{FS} + i\partial\bar{\partial}f^*\phi.$$

Hence, by the Calabi-Yau theorem we have $f^*\phi$ and ϕ differs by a constant, which we denote by c_f. Note that $c_{f^{-1}} = -c_f$, and therefore

$$\frac{1}{|\mathcal{S}|}\sum_{f\in\mathcal{S}} f^*\phi = \frac{1}{|\mathcal{S}|}\sum_{f\in\mathcal{S}}\phi + c_f = \phi + \frac{1}{|\mathcal{S}|}\sum_{f\in\mathcal{S}} c_f = \phi.$$

For an arbitrary element $h \in \mathcal{S}$, then

$$h^*\phi = \frac{1}{|\mathcal{S}|}\sum_{f\in\mathcal{S}} h^*(f^*\phi) = \frac{1}{|\mathcal{S}|}\sum_{f\in\mathcal{S}}(f\circ h)^*\phi = \frac{1}{|\mathcal{S}|}\sum_{\tilde{f}\in h\cdot\mathcal{S}}\tilde{f}^*\phi = \phi,$$

where the last equality is due to $\mathcal{S} \simeq h \cdot \mathcal{S}$. □

Appendix B. Computing volume predictions

For each of the experiments, we check the global consistency of the prediction by computing the volume. This requires numeric integration, which can be performed using Monte-Carlo methods. We follow the procedure of the `cymetric` package, which is described in [22]. Thus, over a sample of points, the integral of a function f is computed as the weighted sum

$$\int_X \mathrm{dVol}_{\mathrm{CY}}\, f = \int_X \mathrm{d}A\, \frac{\mathrm{dVol}_{\mathrm{CY}}}{\mathrm{d}A}\, f \simeq \frac{1}{N}\sum_{i=1}^{N} w_i f(p_i)\ , \tag{B1}$$

where dA is the Fubini–Study measure, and the weights of each point are defined as

$$w_i = \left.\frac{\mathrm{dVol}_{\mathrm{CY}}}{\mathrm{d}A}\right|_{p_i} = \left.\frac{\Omega\wedge\overline{\Omega}}{\mathrm{d}A}\right|_{p_i}\ . \tag{B2}$$

These weights are computed by the point generator routines of the `cymetric` package.

In order to check the global accuracy of the learned metric, we will instead compute

$$\int_X \mathrm{dVol}_{\mathrm{CY}}\, f \simeq \frac{1}{N}\sum_{i=1}^{N} \tilde{w}_i \det\left(g_{\mathrm{CY}}\left(p_i\right)\right) f\left(p_i\right) \ , \tag{B3}$$

using the auxiliary weights $\tilde{w}_i$, defined as $\tilde{w}_i = w_i/(\Omega\wedge\overline{\Omega})|_{p_i}$.

Appendix C. Computing error measures

For each experiment, we check the accuracy of the prediction by computing the σ and Ricci measure which are given by

$$\sigma = \frac{1}{\mathrm{Vol}_{\mathrm{CY}}}\int_X \left\| 1 - \frac{1}{\kappa}\frac{J\wedge J\wedge J}{\Omega\wedge\overline{\Omega}}\right\|_1 , \tag{C1}$$

where $\mathrm{Vol}_{\mathrm{CY}} = \int_X \mathrm{dVol}_{\mathrm{CY}}$ and $\kappa = \left(\int_X J\wedge J\wedge J\right)/\mathrm{Vol}_{\mathrm{CY}}$, as well as

$$\mathcal{R} = \frac{\left(\int_X J\wedge J\wedge J\right)^{1/3}}{\mathrm{Vol}_{\mathrm{CY}}}\int_X \left\|\partial\bar{\partial}\ln\det\left(g_{\mathrm{CY}}\right)\right\|_1 . \tag{C2}$$

Following the default settings of the `cymetric` package, the integrals are numerically evaluated as in equation (B1), except for the integral over the Ricci-scalar which is evaluated as in equation (B3).

Appendix D. Computational performance

Some of the experiments were run on an Intel® Core™ i7-10 700 K CPU with 8 cores. 64 GB of RAM were available during all experiments. Other experiments were run on clusters provided by NAISS, using either different CPU configurations or a GPU.

The homogeneous layer combined with the permutation and root scaling layer trained on 90 000 points for 50 epochs used a maximum of 11 GB of RAM and took approximately 45 min to train. This RAM usage is higher than for the dense layer, and running experiments on a standard laptop with 16 GB of RAM would often crash due to lack of memory. For comparison, the runtimes reported for the dense `cymetric` models in references [21, 22, 27] were about one hour on a laptop.

ORCID iDs

Yacoub Hendi https://orcid.org/0009-0004-5368-8504
Magdalena Larfors https://orcid.org/0000-0003-1011-8777
Moritz Walden https://orcid.org/0000-0001-9900-6485

References

[1] Yau S-T 1978 On the ricci curvature of a compact kähler manifold and the complex monge-ampére equation, I *Commun. Pure Appl. Math.* **31** 339–411
[2] Kachru S, Tripathy A and Zimet M K3 metrics from little string theory (arXiv:1810.10540)
[3] Kachru S, Tripathy A and Zimet M K3 metrics (arXiv:2006.02435)
[4] Tian G 1990 On a set of polarized Kähler metrics on algebraic manifolds *J. Differ. Geom.* **32** 99–130
[5] Donaldson S K 2009 Some numerical results in complex differential geometry *Pure Appl. Math. Q.* **5** 571–618
[6] Douglas M R, Karp R L, Lukic S and Reinbacher R 2007 Numerical solution to the hermitian Yang-Mills equation on the Fermat quintic *J. High Energy Phys.* **12** 083
[7] Douglas M R, Karp R L, Lukic S and Reinbacher R 2008 Numerical Calabi-Yau metrics *J. Math. Phys.* **49** 032302
[8] Braun V, Brelidze T, Douglas M R and Ovrut B A 2008 Calabi-Yau metrics for quotients and complete intersections *J. High Energy Phys.* **05** 080
[9] Headrick M and Wiseman T 2005 Numerical Ricci-flat metrics on K3 *Class. Quantum Grav.* **22** 4931–60
[10] Headrick M and Nassar A 2013 Energy functionals for Calabi-Yau metrics *Adv. Theor. Math. Phys.* **17** 867–902
[11] Cui W and Gray J 2020 Numerical metrics, curvature expansions and Calabi-Yau manifolds *J. High Energy Phys.* **05** 044
[12] Cybenko G 1989 Approximation by superpositions of a sigmoidal function *Math. Control Signals Syst.* **2** 303–14
[13] Leshno M, Lin V Y, Pinkus A and Schocken S 1993 Multilayer feedforward networks with a nonpolynomial activation function can approximate any function *Neural Netw.* **6** 861–7
[14] Maiorov V 2006 Approximation by neural networks and learning theory *J. Complexity* **22** 102–17
[15] Ashmore A, He Y-H and Ovrut B A 2020 Machine learning Calabi–Yau metrics *Fortsch. Phys.* **68** 2000068
[16] Anderson L B, Gerdes M, Gray J, Krippendorf S, Raghuram N and Ruehle F 2021 Moduli-dependent Calabi-Yau and SU(3)-structure metrics from Machine Learning *J. High Energy Phys.* **05** 013

[17] Jejjala V, Mayorga Pena D K and Mishra C 2022 Neural network approximations for Calabi-Yau metrics *J. High Energy Phys.* **2022** 105
[18] Douglas M R, Lakshminarasimhan S and Qi Y Numerical Calabi-Yau metrics from holomorphic networks (arXiv:2012.04797)
[19] Douglas M R 2022 Holomorphic feedforward networks *Pure Appl. Math. Quart.* **18** 251–68
[20] Ashmore A, Calmon L, He Y-H and Ovrut B A 2023 Calabi-Yau Metrics, energy functionals and machine-learning *Int. J. Data Sci. Math. Sci.* **1** 49–61
[21] Larfors M, Lukas A, Ruehle F and Schneider R 2021 Learning size and shape of Calabi-Yau spaces *Machine Learning and the Physical Sciences, Workshop at 35th NeurIPS* (arXiv:2111.01436)
[22] Larfors M, Lukas A, Ruehle F and Schneider R 2022 Numerical metrics for complete intersection and Kreuzer–Skarke Calabi–Yau manifolds *Mach. Learn. Sci. Tech.* **3** 035014
[23] Gerdes M and Krippendorf S 2023 CYJAX: a package for Calabi-Yau metrics with JAX *Mach. Learn.: Sci. Technol.* **4** 025031
[24] Berglund P, Butbaia G, Hübsch T, Jejjala V, Mayorga Peña D, Mishra C and Tan J 2023 Machine Learned Calabi-Yau Metrics and curvature *Adv. Theor. Math. Phys.* **27** 1107–58
[25] Ahmed H and Ruehle F 2023 Level crossings, attractor points and complex multiplication *J. High Energy Phys.* **06** 164
[26] Halverson J and Ruehle F 2024 Metric flows with neural networks *Mach. Learn.: Sci. Technol.* **5** 045020
[27] Anderson L B, Gray J and Larfors M 2023 Lectures on numerical and machine learning methods for approximating Ricci-flat Calabi-Yau metrics (arXiv:2312.17125)
[28] Bronstein M M, Bruna J, LeCun Y, Szlam A and Vandergheynst P 2017 Geometric deep learning: going beyond euclidean data *IEEE Signal Process. Mag.* **34** 18–42
[29] Bronstein M M, Bruna J, Cohen T and Velickovic P 2021 Geometric deep learning: grids, groups, graphs, geodesics and gauges (arXiv:2104.13478)
[30] Cheng M C N, Anagiannis V, Weiler M, de Haan P, Cohen T S and Welling M Covariance in physics and convolutional neural networks (arXiv:1906.02481)
[31] Yarotsky D 2022 Universal approximations of invariant maps by neural networks *Const. Approx.* **55** 407–74
[32] Butbaia G, Mayorga Peña D, Tan J, Berglund P, Hübsch T, Jejjala V and Mishra C 2024 Physical Yukawa couplings in heterotic string compactifications *Adv.Theor. Math. Phys.* **28** (arXiv:2401.15078)
[33] Constantin A, Fraser-Taliente C S, Harvey T R, Lukas A and Ovrut B 2025 Computation of quark masses from string theory *Nucl. Phys. B* **1010**
[34] Candelas P, Dale A M, Lutken C A and Schimmrigk R 1988 Complete Intersection Calabi-Yau Manifolds *Nucl. Phys.* **B298** 493
[35] Aslan B, Platt D and Sheard D 2023 Group invariant machine learning by fundamental domain projections *1st NeurIPS Workshop on Symmetry and Geometry in Neural Representations, PMLR* vol 197 pp 181–218
[36] Kaba S-O, Mondal A K, Zhang Y, Bengio Y and Ravanbakhsh S 2023 Equivariance with learned canonicalization functions *Int. Conf. on Machine Learning* (PMLR) pp 15546–66
[37] Ashmore A and Ruehle F 2021 Moduli-dependent KK towers and the swampland distance conjecture on the quintic Calabi-Yau manifold *Phys. Rev.* D **103** 106028
[38] Douglas M R, Platt D and Qi Y Harmonic 1-forms on real loci of Calabi-Yau manifolds (arXiv:2405.19402)
[39] Raissi M, Perdikaris P and Karniadakis G 2019 Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear parti al differential equations *J. Comput. Phys.* **378** 686–707
[40] Schneider R 2022 Heterotic compactifications in the ERA of data science *PhD Thesis* Uppsala University
[41] Chen S, Dobriban E and Lee J H 2020 A group-theoretic framework for data augmentation *J. Mach. Learn. Res.* **21** 1–71
[42] Fukushima K and Miyake S 1982 Neocognitron: a new algorithm for pattern recognition tolerant of deformations and shifts in position *Pattern Recognit.* **15** 455–69
[43] Lecun Y, Bottou L, Bengio Y and Haffner P 1998 Gradient-based learning applied to document recognition *Proc. IEEE* vol 86 pp 2278–324
[44] Cohen T S and Welling M Group equivariant convolutional networks (arXiv:1602.07576)
[45] Cohen T S, Geiger M, Koehler J and Welling M Spherical CNNs (arXiv:1801.10130)
[46] Gens R and Domingos P M 2014 Deep symmetry networks *Advances in Neural Information Processing Systems* vol 27, ed Z Ghahramani, M Welling, C Cortes, N Lawrence and K Weinberger (Curran Associates, Inc)
[47] Ravanbakhsh S, Schneider J and Poczos B Equivariance through parameter-sharing (arXiv:1702.08389)
[48] Maron H, Litany O, Chechik G and Fetaya E On learning sets of symmetric elements (arXiv:2002.08599)
[49] Gerken J E, Aronsson J, Carlsson O, Linander H, Ohlsson F, Petersson C and Persson D 2023 Geometric deep learning and equivariant neural networks *Artif. Intell. Rev.* 56 14605–62
[50] Green P S, Hubsch T and Lutken C A 1989 All hodge numbers of all complete intersection Calabi-Yau manifolds *Class. Quant. Grav.* **6** 105–24
[51] Lukas A The list of complete intersection calabi-yau three-folds (available at: www-thphys.physics.ox.ac.uk/projects/CalabiYau/cicylist/)
[52] Braun V 2011 On free quotients of complete intersection Calabi-Yau manifolds *J. High Energy Phys.* **04** 005
[53] Lukas A and Mishra C 2020 Discrete symmetries of complete intersection Calabi-Yau manifolds *Commun. Math. Phys.* **379** 847–65
[54] Candelas P and Mishra C 2018 Highly symmetric quintic quotients *Fortsch. Phys.* **66** 1800017
[55] Kreuzer M and Skarke H 2002 Complete classification of reflexive polyhedra in four-dimensions *Adv. Theor. Math. Phys.* **4** 1209–30
[56] Green M B, Schwarz J H and Witten E 2012 *Superstring Theory: Volume 2, Loop Amplitudes, Anomalies and Phenomenology* (Cambridge University Press)
[57] Candelas P, Horowitz G T, Strominger A and Witten E 1985 Vacuum configurations for superstrings *Nucl. Phys.* B **258** 46–74
[58] Witten E 1985 Symmetry breaking patterns in superstring models *Nucl. Phys.* B **258** 75–100
[59] Breit J D, Ovrut B A and Segré G C 1985 E_6 symmetry breaking in the superstring theory *Phys. Lett.* B **158** 33–39
[60] Decker W, Greuel G-M, Pfister G and Schönemann H 2022 SINGULAR 4-3-0 — a computer algebra system for polynomial computations (available at: http://www.singular.uni-kl.de)
[61] Kaplan J, McCandlish S, Henighan T, Brown T B, Chess B, Child R, Gray S, Radford A, Wu J and Amodei D 2020 Scaling laws for neural language models (arXiv:2001.08361)
[62] Ashmore A, Deen R, He Y-H and Ovrut B A 2022 Machine learning line bundle connections *Phys. Lett.* B **827** 136972
[63] Nakahara M 2003 *Geometry, Topology and Physics* (Hilger) p 505

The Geometry of Polynomial Group Convolutional Neural Networks

Yacoub Hendi, Daniel Persson, Magdalena Larfors

February 5, 2026

Abstract

In this article, we study polynomial group convolutional neural networks (PGCNN) for a finite arbitrary group G. In particular, we propose a mathematical framework, using graded group algebras, to describe PGCNN, and we compute the dimension of their neuromanifold, i.e. the space of functions parametrized by this architecture. Moreover, we give a conjecture on the shape of the general fiber for PGCNN.

Contents

1 Introduction **1**

2 Preliminaries on graded group algebras **3**
2.1 Graded group algebras and regular representations . . . 3
2.2 Kronecker and Hadamard products . . . 5
2.3 General conditions and symbolic ranks . . . 7
2.4 Invertibility of filters . . . 7

3 Polynomial group convolutional neural networks **8**
3.1 Definition of polynomial group convolutional neural networks . . . 8
3.2 Neuromanifolds of PGCNNs . . . 9

4 Main results on neuromanifolds of PGCNN **10**
4.1 Dimension of $\mathcal{M}_\varphi$ and $\mathcal{M}_\Phi$ . . . 10
4.2 General fiber of φ . . . 11
4.3 General fiber of Φ . . . 12

5 Conclusion and Outlook **13**

A Proof of Proposition 4.2 **14**

B Proof of Proposition 4.4 **17**

1 Introduction

Geometric deep learning is the subfield in deep learning that studies equivariant architectures with respect to a given group G. The logic behind geometric deep learning is that when training over a dataset that obeys a certain set of symmetries, we can introduce an informed bias towards equivariant and invariant models by using equivariant architectures. The benefit of doing this is a reduction of sample complexity; this can be seen since an equivariant architecture has fewer parameters due to weight-sharing, and fewer parameters need fewer training data points to determine their values and, henceforth, smaller sample complexity [DWZ+18]. A classical example of equivariant architectures is convolutional neural networks (CNN), which are equivariant under discrete translations, corresponding to the group $G = \mathbb{Z}$. The generalization to arbitrary finite groups is presented in [CW16, CGW19]. For a more extensive account about geometric deep learning, see the book [BBCV21].

Broadly speaking, the main questions addressed in geometric deep learning lie in two main categories. The first one focuses on design principles and construction techniques used to build equivariant architectures for different groups. The second one is concerned with the expressivity

and training dynamics of such models. In this article we are interested in the latter direction, and to investigate this, we apply tools from the recently emergent field of *neuroalgebraic geometry* [MSM+25].

The main idea of neuroalgebraic geometry can be briefly understood as follows. Given a fixed neural network architecture Φ, one obtains a parametric family of functions Φ_θ indexed by the network parameters θ. We call the space of all realizable functions *the neuromanifold* $\mathcal{M}_\Phi$ associated with the architecture Φ. The term "manifold" is a slight misnomer, as these functional spaces usually exhibit singularities. Given a loss function $\mathcal{L}$ and a dataset $\mathcal{D}$, then learning via stochastic gradient descent can be interpreted as a distance minimization problem over the neuromanifold to an objective function $f_{\mathcal{L},\mathcal{D}}$ in the ambient space, that depends on both the loss function $\mathcal{L}$ and the dataset $\mathcal{D}$. From this perspective, the geometric properties of the neuromanifold translate into statistical counterparts. For example, dimension and curvature translate into sample complexity and expressivity, fiber classification translates into identifiability, and so on.

However, the analysis of these neuromanifolds is often intractable due to their compositional complexity and the use of nonlinear activation functions, which in some cases fail to be analytic such as the RELU function. One useful simplification is to restrict to polynomial activation functions. Then the neuromanifold is a subspace of the space of homogeneous polynomials of fixed degree and in a fixed number of variables. More precisely, the neuromanifold becomes a semi-algebraic set (i.e. it can be described as a solution set of polynomial equations and inequalities in its finite dimensional ambient space), which can be analyzed using tools from algebraic geometry.

In this paper, we focus on the study of polynomial group convolutional neural networks (PGCNN). We formalize the construction of PGCNNs using the language of group algebras and describe the neuromanifolds as subspaces of these algebras. The aim of this paper is to generalize the results in [SMK24] which hold for polynomial CNNs to PGCNN for an arbitrary finite group G.

Our contribution.

1. We propose a mathematical framework for polynomial group convolutional networks in the language of graded group algebras. See §2 and §3.

2. More importantly, we provide two ways to describe a polynomial activation function of degree r in this language. It can either be described by the Kronecker product or the Hadamard product. Depending on which we choose, we get two different parametrization maps denoted by φ and Φ, respectively. These two maps are related by $\Phi = \Lambda \circ \varphi$ where Λ is a linear map.

3. For the polynomial group convolutional architecture of depth $L \geq 1$ and a polynomial activation function of degree $r \geq 2$ associated with a finite group G, the neuromanifold $\mathcal{M}_\Phi$ and the image $\mathcal{M}_\varphi$ of φ have the same dimension $L(|G| - 1) + 1$; see Theorem 4.3 and Theorem 4.1. Note that the dimension does not depend on the degree r or the particular group structure of G but only on the size of G and the number of layers L.

4. From the previous point, it follows directly that the general fibers of the parametrization maps Φ and φ are finite, up to scaling. Moreover, we give a precise description of the general fiber of φ, see Theorem 4.6, and conjecture that the same description holds for the general fiber of Φ.

It is worth highlighting here that the results from [MM25, UBDC25, FRWY25] show that the neuromanifolds of PNNs with fixed width have the expected (maximum) dimension. More precisely, for a *general element* of such a neuromanifold, the tangent space has the expected dimension. Since a PGCNN can be represented as a PNN with fixed width, then one might expect that the dimension of the neuromanifold of PGCNN can be inferred immediately from these results. Nevertheless, this is not true because a PGCNN enforces weight-sharing constraints that violate the required generality assumption.

Related works. We review here works related to neuroalgebraic geometry and focus on those concerning convolutional and equivariant architectures.

Mehta et al. in [MCTH18] presented one of the first attempts to use algebraic geometry to count critical points in the loss landscape for deep linear neural networks. Building on this perspective, Traget et al. in [TKB20] introduced the classification of critical points as spurious (due to parametrization) and pure (due to geometry). In [KTB19], Kileel et al. connected the dimension of the neuromanifold of PNNs to the expressivity of the model and established upper

and lower bounds on layer width for neuromanifolds to be filling (i.e. they fill their ambient space). In the same paper, they conjectured that for sufficiently high activation degrees, all PNN architectures are filling. This conjecture was later proven independently in both [FRWY25] and [MM25]. Furthermore, questions about non-defectiveness and identifiability of PNNs were settled separately by Usevich et al. in [UBDC25] using techniques from tensor decompositions and by Massarenti and Mella in [MM25] by analyzing secant varieties.

Turning to convolutional architectures, the neuromanifold of a linear convolutional network was first studied in [KMMT22] and [KMST24]. In these works, convolution filters were identified with homogeneous bivariate polynomials and the convolution operation with polynomial multiplication. These allowed the authors to define the neuromanifold by factorization conditions and to show it is a semi-algebraic set; moreover, conditions under which it is filling are established. Building on this line of work, Shahverdi [Sha24] developed recursive algorithms to construct the polynomial equations and inequalities that define the neural manifold for a linear convolutional network, and it relates the number of critical points of the square loss function to the generic Euclidean degree of the Segre variety.

The work most closely related to this article is [SMK24]. In this work, Shahverdi et al. compute the dimension, the degree and the generic Euclidean degree for polynomial convolutional networks in addition to classifying the general fiber and the singular locus. A central aspect of their analysis is based on the relation between the neuromanifold and the Segre-Veronese; in particular, they show that the neuromanifold for CNN is birational to the Segre-Veronese variety. Our work generalizes their dimension result for PGCNNs.

Notation. We denote by $\mathbb{K}$ an infinite field with characteristic 0 and equipped with a nontrivial absolute value, for example $\mathbb{K}$ can be the real numbers, the complex numbers, the rational numbers or p-adic numbers. For a field $\mathbb{K}$ and n variables $x = (x_1, \dots, x_n)$ we denote the ring of polynomials in x over the field $\mathbb{K}$ by $\mathbb{K}[x_1, \dots, x_n]$ and the space of homogeneous polynomials in the variables x of degree d by $\mathrm{Sym}_{\mathbb{K}}(x, d)$. More generally, let the variables be partitioned into m blocks $x^{(1)}, \dots, x^{(m)}$, where $x^{(i)} = (x_1^{(i)}, \dots, x_{n_i}^{(i)})$. A polynomial $f \in \mathbb{K}[x^{(1)}, \dots, x^{(m)}]$ is called *multihomogeneous* of multidegree $(d_1, \dots, d_m)$ if it is homogeneous of degree d_i with respect to each block $x^{(i)}$. We denote the space of such polynomials by $\mathrm{Sym}_{\mathbb{K}}\big((x^{(1)}, \dots, x^{(m)}), (d_1, \dots, d_m)\big)$.

2 Preliminaries on graded group algebras

In this section, we present graded group algebras, their regular representations, and how Kronecker and Hadamard products act on them. These are the building blocks for polynomial group convolutional networks. The section can be skipped, in case the reader is familiar with these concepts from algebra, and returned to later when needed.

In the following, G and H are general finite groups with order $n = |G|$ and $|H|$ and $S := \bigoplus_i S_i$ is a graded commutative ring such as the ring of polynomials in n variables $\mathbb{K}[x_1, \dots, x_n]$. For two elements $g, h \in G$ we denote their group product by gh and for two elements $s, t \in S$ we denote their ring product by st and their sum by $s + t$.

2.1 Graded group algebras and regular representations

Here we present graded group algebras and their regular representations, which describe the G action on group convolutional networks. We show, under a fixed choice of basis, these representations are described by G-circulent matrices, which are a generalization of the circulent matrices in the case where $G = \mathbb{Z}_n$.

Definition 2.1. The graded group algebra $S[G]$ is an algebra

$$S[G] := \{\theta \mid \theta : G \to S\} \cong S^{|G|},$$

where $S^{|G|}$ is the free S-module of rank $|G|$. The graded algebra bilinear product in $S[G]$ is given by *the convolution operation,*

$$(\theta * \psi)(g) := \sum_{h \in G} \theta(gh^{-1})\psi(h),$$

and the scalar multiplication is defined componentwise; i.e. for $s \in S$ we define $(s\theta)(g) := s\theta(g)$. A basis of $S[G]$ is given by the group elements in G. We call elements in $S[G]$ S-filters over G.

Remark 2.2. There are multiple equivalent ways to describe a filter $\theta \in S[G]$. In the definition it is described as a function from G to S. Another useful way to describe this filter is to identify it with the formal linear combination

$$\theta \equiv \sum_{g \in G} \theta(g) g.$$

Then the convolution defined above becomes just the product of such formal linear combinations. Note that if $G = (\mathbb{Z}, +)$, then the description of a finite filter as a formal linear combination identifies it with a polynomial in one variable, which is the identification of convolutional filters in linear CNN presented in [KMMT22].
Also from here it is clear how $S[G]$ is isomorphic to the free S-module of rank $|G|$. Lastly, if we fix an order $(g_1, \ldots, g_n)$ on the elements of G, then we can also identify the filter θ with the tuple $(\theta(g_1), \ldots, \theta(g_n))$.

Example 2.3. Let $G = \mathbb{Z}_3 = \{\mathbf{0}, \mathbf{1}, \mathbf{2}\}$ be the cyclic group of order 3 (with addition modulo 3), and let $S = \mathbb{R}$ concentrated in degree 0. Then

$$\mathbb{R}[G] = \{\theta : \mathbb{Z}_3 \to \mathbb{R}\} \cong \mathbb{R}^3.$$

A filter $\theta \in \mathbb{R}[G]$ is determined by its values

$$\theta(\mathbf{0}) = a, \qquad \theta(\mathbf{1}) = b, \qquad \theta(\mathbf{2}) = c,$$

and can be written as the formal linear combination

$$\theta = a\,\mathbf{0} + b\,\mathbf{1} + c\,\mathbf{2}.$$

Similarly, let $\psi \in \mathbb{R}[G]$ be given by

$$\psi(\mathbf{0}) = x, \qquad \psi(\mathbf{1}) = y, \qquad \psi(\mathbf{2}) = z, \quad \text{so that} \quad \psi = x\,\mathbf{0} + y\,\mathbf{1} + z\,\mathbf{2}.$$

The convolution $\theta * \psi$ is the function $\mathbb{Z}_3 \to \mathbb{R}$ defined by

$$(\theta * \psi)(g) = \sum_{h \in \mathbb{Z}_3} \theta(g - h)\psi(h),$$

where subtraction is taken modulo 3. Explicitly,

$$\begin{aligned}
(\theta * \psi)(\mathbf{0}) &= ax + bz + cy, \\
(\theta * \psi)(\mathbf{1}) &= ay + bx + cz, \\
(\theta * \psi)(\mathbf{2}) &= az + by + cx.
\end{aligned}$$

Definition 2.4 ($S[G]$ regular representation)**.** The regular algebra representation ρ of $S[G]$ on itself is defined as

$$\begin{aligned}
\rho : S[G] &\to \mathrm{End}(S[G]), \qquad \rho_\theta(\psi) = \theta * \psi. \\
\theta &\mapsto \rho_\theta
\end{aligned}$$

The algebra representation ρ is completely determined by the induced group representation of G on $S[G]$.

Definition 2.5 (G-circulent matrix)**.** Fix an order $(g_1, \ldots, g_n)$ of the elements of G. With respect to this basis, the map ρ_θ is represented by an $n \times n$ *G-circulent matrix* Mat_θ over S, whose entries are given by

$$(\mathrm{Mat}_\theta)_{ij} := \theta(g_i g_j^{-1}).$$

So the vector representation of $\mathrm{Mat}_\theta \psi$ under this base is given by

$$(\mathrm{Mat}_\theta \psi)_i = \sum_{j=1}^{n} (\mathrm{Mat}_\theta)_{ij} \psi(g_j),$$

which is just a rewriting of the convolution $(\theta * \psi)(g_i)$. In the special case $G = \mathbb{Z}_n$, the matrix Mat_θ reduces to the classical circulent matrix.

Remark 2.6. Note that since Mat is the algebra representation ρ under a choice of base, it follows immediately that for any filters $\theta, \psi \in S[G]$ we get

$$\mathrm{Mat}_\theta \mathrm{Mat}_\psi = \mathrm{Mat}_{\theta * \psi} \quad \text{and} \quad \mathrm{Mat}_\theta + \mathrm{Mat}_\psi = \mathrm{Mat}_{\theta+\psi}.$$

Also, the set $\{\mathrm{Mat}_g \mid g \in G\}$ defines a basis for G-circulent matrices over the algebra S, where we identify $g \in G$ with the filter $g(h) = 1$ if $h = g$ otherwise $g(h) = 0$.

In the following proposition, we relate the invertibility of G-circulent matrices to the invertibility of filters as elements in the group algebra $S[G]$.

Proposition 2.7. *Let $\theta \in S[G]$. If Mat_θ is invertible, then its inverse is also G-circulent. By Remark 2.6, Mat_θ is invertible if and only if θ is invertible in $S[G]$.*

Proof. Since S is commutative, we can apply Cayley-Hamilton to the inverse Mat_θ^{-1} to write it as a polynomial in Mat_θ. Since $\mathrm{Mat}_\theta = \sum_{g \in G} \theta(g)\mathrm{Mat}_g$, then Mat_θ^{-1} is also a linear combination of the matrices Mat_g's with coefficients in S. Therefore Mat_θ^{-1} is also G-circulent. □

2.2 Kronecker and Hadamard products

Next we define the Kronecker product and the Hadamard product between filters and state propositions about their interactions with convolution. These two operations will give us two ways to define polynomial activation functions.

Proposition 2.8 (Definition of Kronecker Product)**.** *There is a canonical isomorphism ι between $S[G] \otimes S[H]$ and $S[G \times H]$ given by*

$$\begin{aligned} \iota : S[G] \otimes S[H] &\to S[G \times H] \\ \theta \otimes \psi &\mapsto \iota(\theta \otimes \psi) \end{aligned}$$

where for $(g, h) \in G \times H$, $\iota(\theta \otimes \psi)(g, h) := \theta(g)\psi(h)$. By abuse of notation, we denote the filter $\iota(\theta \otimes \psi)$ by $\theta \otimes \psi$ and call it the Kronecker product of θ and ψ.

Remark 2.9. For $r \geq 1$, and $\theta \in S[G]$ we use the notation

$$\theta^{\otimes r} := \underbrace{\theta \otimes \ldots \otimes \theta}_{r},$$

which is a filter in $S[G^r]$ where G^r is the direct product of r copies of G. Note that the Kronecker product respects the grading; i.e. if $\theta \in S_k[G]$ and $\psi \in S_l[H]$ then $\theta \otimes \psi \in S_{k+l}[G \times H]$ and $\theta^{\otimes r} \in S_{rk}[G^r]$.

Definition 2.10 (Hadamard Product)**.** Let $\theta, \psi \in S[G]$, their *Hadamard product* is the filter denoted by $\theta \odot \psi \in S[G]$ and is given by

$$(\theta \odot \psi)(g) := \theta(g)\psi(g).$$

Moreover, we denote the rth Hadamard power of θ by $\sigma_r(\theta) := \underbrace{\theta \odot \ldots \odot \theta}_{r}$. The Hadamard product also respects the grading of S.

To justify our terminology, note that for any two filters θ and ψ we have the identities $\mathrm{Mat}_{\theta \otimes \psi} = \mathrm{Mat}_\theta \otimes \mathrm{Mat}_\psi$ and $\mathrm{Mat}_{\theta \odot \psi} = \mathrm{Mat}_\theta \odot \mathrm{Mat}_\psi$ where on the right hand side in both of them, we use the Kronecker product and the Hadamard product between matrices, respectively.

Example 2.11. Let $G = H = \mathbb{Z}_2 = \{0, 1\}$ and $S = \mathbb{R}$. Fix the ordered bases $\{0, 1\}$ of $\mathbb{R}[G]$ and $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$ of $\mathbb{R}[G \times H]$.

Let $\theta \in \mathbb{R}[G]$ and $\psi \in \mathbb{R}[H]$ be given by

$$\theta(0) = a,\ \theta(1) = b, \qquad \psi(0) = x,\ \psi(1) = y.$$

A direct computation shows that left convolution by θ and ψ is represented by the matrices

$$\mathrm{Mat}_\theta = \begin{pmatrix} a & b \\ b & a \end{pmatrix}, \qquad \mathrm{Mat}_\psi = \begin{pmatrix} x & y \\ y & x \end{pmatrix}.$$

The Kronecker product of these matrices is

$$\mathrm{Mat}_\theta \otimes \mathrm{Mat}_\psi = \begin{pmatrix} ax & ay & bx & by \\ ay & ax & by & bx \\ bx & by & ax & ay \\ by & bx & ay & ax \end{pmatrix}.$$

On the other hand, the filter $\theta \otimes \psi \in \mathbb{R}[G \times H]$ satisfies $(\theta \otimes \psi)(g,h) = \theta(g)\psi(h)$, and convolution by $\theta \otimes \psi$ on $\mathbb{R}[G \times H]$ is represented, in the chosen basis, by the same matrix.

The following proposition shows that the Kronecker product distributes over convolution.

Proposition 2.12. *Let $\theta_1, \theta_2 \in S[G]$ and $\psi_1, \psi_2 \in S[H]$. Then $(\theta_1 \otimes \psi_1) * (\theta_2 \otimes \psi_2) = (\theta_1 * \theta_2) \otimes (\psi_1 * \psi_2)$. By induction it holds that for $\theta_i, \psi_i \in S[G_i]$ for $1 \le i \le r$*

$$(\theta_1 \otimes \ldots \otimes \theta_r) * (\psi_1 \otimes \ldots \otimes \psi_r) = (\theta_1 * \psi_1) \otimes \ldots \otimes (\theta_r * \psi_r).$$

Proof. Use definition of convolution, we get

$$\begin{aligned}
(\theta_1 \otimes \psi_1) * (\theta_2 \otimes \psi_2)(g,h) &= \sum_{g',h'} (\theta_1 \otimes \psi_1)(gg'^{-1}, hh'^{-1})(\theta_2 \otimes \psi_2)(g',h') \\
\text{(use definition of } \otimes) &= \sum_{g',h'} \theta_1(gg'^{-1})\psi_1(hh'^{-1})\theta_2(g')\psi_2(h') \\
\text{(reorder the terms)} &= \sum_{g',h'} \theta_1(gg'^{-1})\theta_2(g')\psi_1(hh'^{-1})\psi_2(h') \\
\text{(split the sum into product of two sums)} &= \left(\sum_{g'} \theta_1(gg'^{-1})\theta_2(g')\right)\left(\sum_{h'} \psi_1(hh'^{-1})\psi_2(h')\right) \\
&= (\theta_1 * \theta_2)(g)(\psi_1 * \psi_2)(h) = (\theta_1 * \theta_2) \otimes (\psi_1 * \psi_2)(g,h)
\end{aligned}$$

□

On the other hand, the Hadamard product does not distribute over convolution, and this is easy to check as follows. Let $\theta_1, \theta_2, \psi_1, \psi_2 \in S[G]$ then

$$\begin{aligned}
(\theta_1 \odot \theta_2) * (\psi_1 \odot \psi_2)(g) &= \sum_{h \in G} \theta_1(gh^{-1})\theta_2(gh^{-1})\psi_1(h)\psi_2(h) \\
&\neq \left(\sum_{h \in G} \theta_1(gh^{-1})\psi_1(h)\right)\left(\sum_{h \in G} \theta_2(gh^{-1})\psi_2(h)\right) = (\theta_1 * \psi_1) \odot (\theta_2 * \psi_2)(g).
\end{aligned}$$

The following proposition explains the relation between the Hadamard product, the Kronecker product, and convolution. This is an important proposition, as it tells us how to move between the two descriptions of polynomial activation functions; see §3.

Proposition 2.13. *Let $\theta_1, \psi_1, \theta_2, \psi_2 \in S[G]$. Then*

$$(\theta_1 * \psi_1) \odot (\theta_2 * \psi_2) = ((\theta_1 \otimes \theta_2) * (\psi_1 \otimes \psi_2))\,|_G.$$

It follows that for $\theta, \psi \in S[G]$ and $r \ge 1$ we get $\sigma_r(\theta * \psi) = (\theta^{\otimes r} * \psi^{\otimes r})|_G$, where G is identified with the diagonal subgroup in G^r.

Proof. By Proposition 2.12,

$$(\theta_1 \otimes \theta_2) * (\psi_1 \otimes \psi_2) = (\theta_1 * \psi_1) \otimes (\theta_2 * \psi_2).$$

Restricting the right hand side to the diagonal subgroup $G < G^2$ gives us $(\theta_1 * \psi_1) \odot (\theta_2 * \psi_2)$, as required. □

Remark 2.14. If H is a subgroup of G then $S[H]$ is a subalgebra of $S[G]$. Therefore, for $\psi \in S[H]$ and $\theta \in S[G]$ we have $\theta * \psi, \psi * \theta \in S[G]$. Namely,

$$(\theta * \psi)(g) = \sum_{h \in H} \theta(gh^{-1})\psi(h) \text{ and } (\psi * \theta)(g) = \sum_{h \in H} \psi(h)\theta(h^{-1}g).$$

In other words, any filter $\psi \in S[H]$ can be extended to G by setting $\psi(g) = 0$ for all $g \notin H$. In case we need to distinguish between $\psi \in S[H]$ and its extension $\psi \in S[G]$ we denote the extension by ψ^G and its associated matrix by Mat_ψ^G.

2.3 General conditions and symbolic ranks

To compute the dimension of the neuromanifold, which is the image of the parametrization map Φ (see §3), we will need to compute the rank of the Jacobian of the parametrization map Φ at a general set of parameters θ. In this subsection, we define what a general condition is, and we introduce the concept of symbolic matrix and symbolic ranks that will help us define general conditions.

Definition 2.15 (generality)**.** We say that a condition $\mathcal{P}$ is *general* in $\mathbb{K}^m$ if it holds over a dense set in $\mathbb{K}^m$ equipped with the Zariski topology.

Another more intuitive way to understand generality, although not exactly equivalent, is to think of $\mathcal{P}$ as holding everywhere in $\mathbb{K}^m$ except for a set of Lebesgue measure zero.

Definition 2.16. A symbolic matrix M is a matrix over a ring of polynomials $\mathbb{K}[y_1, \ldots, y_m]$. We say M is symbolic in terms of the indeterminates $(y_1, \ldots, y_m)$. The symbolic rank of M is decided by the vanishing of its minors in the ring $\mathbb{K}[y_1, \ldots, y_m]$.

For a symbolic matrix M over $\mathbb{K}[y_1, \ldots, y_m]$ and a point $p = (p_1, \ldots, p_m) \in \mathbb{K}^m$, we construct the matrix $M(p)$ from M by substituting every y_i for p_i. Then the condition on p to satisfy $\operatorname{rank}(M) = \operatorname{rank}(M(p))$ is general in $\mathbb{K}^m$, because this condition fails if and only if p is the zero of a nontrivial minor of M. We illustrate this in the following example.

Example 2.17. Consider the symbolic matrix over $\mathbb{K}[y_1, y_2]$

$$M = \begin{pmatrix} y_1 & y_2 \\ y_2 & y_1 \end{pmatrix}.$$

The determinant of M is the polynomial

$$\det M = y_1^2 - y_2^2.$$

Hence, over the polynomial ring $\mathbb{K}[y_1, y_2]$, the symbolic rank of M is 2since the determinant is not the zero polynomial in $\mathbb{K}[y_1, y_2]$. Therefore, a point $p = (p_1, p_2) \in \mathbb{K}^2$ satisfies $\operatorname{rank}(M) = \operatorname{rank}(M(p_1, p_2))$ if and only if $p_1^2 - p_2^2 \neq 0$ which is a general condition in $\mathbb{K}^2$. This example illustrates that the rank of a symbolic matrix is determined by polynomial conditions on the indeterminates and that rank deficiencies occur precisely on algebraic varieties defined by the vanishing of suitable minors.

2.4 Invertibility of filters

In this subsection, a general filter $\theta \in S[G]$ is shown to be invertible in the group algebra. Similar results hold for its extension to G^r, its rth Hadamard product, and its rth Kronecker product. These results will be relevant in the proof of the main theorems. Since we also work in a general setting, we will be allowed to assume that the matrices we work with are full rank.

To comment on what is aimed for in these proofs, note that a general square matrix M has, of course, a full rank because its determinant defines a nontrivial polynomial in terms of its entries. What we show in what follows is that for G-circulent matrices, which have weight-sharing constraints, the determinant still defines a nontrivial polynomial regardless of the group G.

Definition 2.18. For a filter $\theta \in S[G]$, we define its determinant to be the determinant of its associated matrix, i.e. $\det(\theta) := \det(\mathrm{Mat}_\theta)$.

In the following proposition, we relate the determinants of the three filters $\theta \in S[G]$ and $\theta^{\otimes}, \theta^{G^r} \in S[G^r]$.

Proposition 2.19. *Let $\theta \in S[G]$, then we have*

$$\det(\theta^{\otimes r}) = \det(\theta)^{rn^r} \quad \textit{and} \quad \det(\theta^{G^r}) = \det(\theta)^{n^{r-1}}$$

where $n = |G|$. This implies that $\theta \in S[G]$ is invertible if and only if $\theta^{\otimes r}$ and θ^{G^r} in $S[G^r]$ are both invertible.

Proof. The first equality follows directly from the property of the Kronecker product for matrices.

The second equality follows from a standard argument in representation theory. Since we identify G with the diagonal subgroup $\Delta < G^r$, we obtain that the restriction of the regular algebra representation of $S[G^r]$ to $S[G]$ has $|G^r/G| = n^{r-1}$ unique $S[G]$-invariant sub-representation $V_i \subset S[G^r]$ where $S[G^r] = \bigoplus_{i=1}^{n^{r-1}} V_i$ and $S[G] \cong V_i$ for all i. Therefore, under a suitable choice of basis, we have $\mathrm{Mat}_\theta^{G^r}$ is a block diagonal matrix of n^{r-1} blocks where each block is equal to Mat_θ. This proves our claim. □

In the following proposition, we state that invertibility of filters is a general condition in $S[G]$.

Proposition 2.20. *Let $\theta \in S[G]$ then the following conditions are general for all $r \geq 1$:*

1. $\det(\theta) \neq 0$ *2.* $\det(\theta^{G^r}) \neq 0$ *3.* $\det(\theta^{\otimes r}) \neq 0$ *4.* $\det(\sigma_r(\theta)) \neq 0$.

Proof. Assume $G = \{g_1, \ldots, g_n\}$ and $\theta = (\theta_{g_1}, \ldots, \theta_{g_n})$ where $g_1 = e$ is the group identity. We have $(\mathrm{Mat}_\theta)_{i,j} = \theta(g_i g_j^{-1})$. For fixed g_i (row) and g_k there is a unique g_j such that $g_i g_j^{-1} = g_k$, namely $g_j = g_k^{-1} g_i$. Similarly for fixed g_j (column) and g_k there is a unique g_i such that $g_i g_j^{-i} = g_k$. This means the element $\theta(g_k)$ occurs once and only once in each column and in each row in the matrix Mat_θ. Therefore by elementary rules of the determinant, the term $\theta(g_k)^n$ for all $g_k \in G$ occurs once and only once in the determinant of Mat_θ. This implies the condition $\det(\mathrm{Mat}_\theta) = \det(\theta) \neq 0$ in $S[G]$ is general. By Proposition 2.19, the conditions 2 and 3 are general.

The same argument shows that $\det(\sigma_r(\theta))$ is not the trivial polynomial because it includes the terms $\theta(g_k)^{rn}$ for all $g_k \in G$. □

Corollary 2.21. *Propositions 2.19 and 2.20 imply that generally $\theta, \sigma_r(\theta) \in S[G]$ and $\theta^{\otimes r}, \theta^{G^r} \in S[G^r]$ are invertible, for all $r \geq 1$. In particular, they are generally not zero divisors in their algebras.*

3 Polynomial group convolutional neural networks

In this section, polynomial group convolutional networks and their neuromanifolds are defined, which are the main object of study here. Recall that the main goal of this article is to compute the dimension of these neuromanifolds.

3.1 Definition of polynomial group convolutional neural networks

Definition 3.1 (GCNN and PGCNN)**.** A group convolutional neural network (GCNN) with the filters $\theta := (\theta_1, \ldots, \theta_L) \in (\mathbb{K}[G])^L$ and an activation function $\sigma : \mathbb{K} \to \mathbb{K}$ is the map

$$\begin{aligned} \Phi_\theta : \mathbb{K}[G] &\to \mathbb{K}[G] \\ x &\mapsto \sigma(\ldots(\sigma(\sigma(x * \theta_1) * \theta_1) * \ldots * \theta_{L-1}) * \theta_L, \end{aligned}$$

where σ acts component wise. When $\sigma = \sigma_r$, where σ_r denotes the rth Hadamard product, we call Φ_θ a polynomial group convolutional neural network (PGCNN). We call x the input filter and θ_l the lth weight filter for $1 \leq l \leq L$.

The GCNNs are G-equivariant, where the G-action is the one given by the regular representation in Section §2, and equivariance follows directly from the associativity of convolution. Before discussing PGCNN further, we present a few remarks to connect our definition with existing work in the literature.

Remark 3.2. Convolutional neural networks, regardless of their name, are in the literature defined using the operation *cross correlation* $\star$ instead of convolution $*$. Nevertheless, these two operations are intrinsically connected and interchangeable via the relation $\theta \star \psi := \theta * \overline{\psi}$, where the filter $\overline{\psi}(g) := \psi(g^{-1})$ is called the involution of ψ. Therefore these two conventions are equivalent in the sense that every neural network defined via convolution can be defined via cross correlation by involuting all its filters. In this article, we adopt the convolutional convention for two reasons. Firstly, under this convention, the G-action translates to just one extra left convolution, and the equivariance property translates into associativity of the convolution operation. Secondly, by associativity of convolutions, we can study the interaction between the weight filters θ away from the input filter x.

Remark 3.3. In machine learning literature, the input filter x is called the input signal, and the weight filters correspond to convolutional layers. Moreover, the signal x is considered over a domain Ω equipped with a G action. In other words, $x : \Omega \to \mathbb{K}$ where for $t \in \Omega$ and $g \in G$ we have $(g \cdot x)(t) := x(g^{-1} \cdot t)$. In this context, what we define above is a special case of GCNN where the domain $\Omega = G$. This simplification is convenient because it keeps the group algebra structure at the first layer.

Remark 3.4. Notice that when $G = \mathbb{Z}_n$ is the nth cyclic group, the GCNN does *not* specialize to CNN considered, for example, in [KMMT22] and [SMK24] because these CNNs are *not equivariant* with respect to shifts. The main reason is that in this CNN architecture, the domain is considered to be a sequence rather than a circle. To see this, let C_k denotes the domain that is a sequence of length k. Moreover, let x be a signal on C_n and θ be a signal (or filter) over C_k where $k < n$. Then their cross correlation with a step of length 1 will be the signal $x \star_{CNN} \theta$ over C_{n-k+1} given by

$$(x \star_{CNN} \theta)(l) := \sum_{j=0}^{k-1} x(l+j)\theta(j), \text{ where } l < n-k+1.$$

Assume that s_d denotes the shift to the right with d steps; then $(s_d \cdot x)(l) := \theta(l - d \mod n)$, note importantly that the order of the modulo operation depends on the size of the signal x. Given this, we have

$$s_d \cdot (x \star_{CNN} \theta)(l) = (x \star_{CNN} \theta)(l - d \mod (n-k+1)) = \sum_{j=0}^{k-1} x([l - d \mod (n-k+1)] + j)\theta(j),$$

and

$$(s_d \cdot x \star_{CNN} \theta)(l) = \sum_{j=0}^{k-1} s_d \cdot x(j+l)\theta(j) = \sum_{j=0}^{k-1} x([l - d \mod n] + j)\theta(j).$$

This implies $s_d \cdot (x \star_{CNN} \theta) \neq s_d \cdot x \star_{CNN} \theta$, unless $k = 1$ which means the convolution filter contains only one element. Hence, standard discrete CNNs with finite filters are not equivariant with respect to cyclic shifts, except in the trivial case $k = 1$.

3.2 Neuromanifolds of PGCNNs

From Definition 3.1, we can see that for any tuple of L filters $\theta := (\theta_1, \ldots, \theta_L) \in (\mathbb{K}[G])^L$ and the activation function σ_r, the PGCNN $\Phi_\theta \in \mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})[G]$ defines n homogeneous polynomials in n variables $x := (x_1, \ldots, x_n)$ with degree r^{L-1}. Moreover, each coefficient in the homogeneous polynomial $\Phi_\theta(g)$ is itself a multihomogeneous polynomial with the multidegree $\mathbf{r} := (r^{L-1}, r^{L-2}, \ldots, 1)$ in the parameters of the weight filters $\theta = (\theta_1, \ldots, \theta_L)$; namely it belongs to the space $\mathrm{Sym}_{\mathbb{K}}(\theta, \mathbf{r})$.

Definition 3.5 (Neuromanifold)**.** The *neuromanifold* $\mathcal{M}_\Phi$ of the architecture of PGCNN of depth L and activation degree r is the image of the parameterization map $\Phi : \theta \mapsto \Phi_\theta$ in $\mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})$; namely

$$\mathcal{M}_\Phi = \{\Phi_\theta \mid \theta \in (\mathbb{K}[G])^L\} \subset \mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})[G].$$

We define the associated *neurovariety* $\mathcal{V}_\Phi$ to be the Zariski closure of $\mathcal{M}_\Phi$ in $\mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})[G]$.

Note that the projection map $\Phi_\theta \mapsto \Phi_\theta(e)$ over the identity element is injective due to equivariance. Therefore we can consider the ambient space of the neuromanifold $\mathcal{M}_\Phi$ to be $\mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})$ rather than $\mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})[G]$.

As we have seen until now, the activation function is represented using the Hadamard product. Another equivalent way is to replace the Hadamard product with the Kronecker product using Proposition 2.13. So in terms of the Kronecker product, the filter Φ_θ can be written as

$$\Phi_\theta = (x^{\otimes r^{L-1}} * \varphi_\theta)|_G,$$

where $\varphi_\theta = \theta_1^{\otimes r^{L-1}} * \theta_2^{\otimes r^{L-2}} * \ldots * \theta_L \in \mathbb{K}[G^{r^{L-1}}]$. We define the parametrization map

$$\begin{aligned} \varphi : \mathbb{K}[G]^L &\to \mathbb{K}[G^{r^{L-1}}] \\ \theta &\mapsto \varphi_\theta \end{aligned}$$

and we denote its image by $\mathcal{M}_\varphi := \{\varphi_\theta \mid \theta \in \mathbb{K}[G]^L\} \subset \mathbb{K}[G^{r^{L-1}}]$.

$$\begin{tikzcd} \mathbb{P}(\mathbb{K}[G])^L \arrow[r, "\tilde{\varphi}"] \arrow[rr, bend left, "\tilde{\Phi}"] & \mathbb{P}\mathcal{M}_\varphi \subset \mathbb{P}(\mathbb{K}[G^{r^{L-1}}]) \arrow[r, "\Lambda"] & \mathbb{P}\mathcal{M}_\Phi \subset \mathbb{P}(\mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})) \end{tikzcd}$$

Figure 1: A commutative diagram that describes the relation between the projectivized parametrization maps $\tilde{\Phi}$ and $\tilde{\varphi}$ induced by the PGCNN architecture with L layers and activation degree r over the finite group G.

Both of the maps Φ and φ are defined by multihomogeneous polynomials in the weight parameters $\theta = (\theta_1, \ldots, \theta_L)$, and therefore, by the Tarski-Seidenberg Theorem, both $\mathcal{M}_\Phi$ and $\mathcal{M}_\varphi$ are semi-algebraic sets in their respective ambient spaces. Moreover, they induce rational maps $\tilde{\Phi}$ and $\tilde{\varphi}$ on the projectivizations of their domains and codomains. The relation between the rational maps $\tilde{\varphi}$ and $\tilde{\Phi}$ is shown in a commutative diagram, see Figure 1, where Λ is a projective linear map (it consists of convolution with $x^{\otimes r^{L-1}}$ and the restriction to $e \in G$).

Remark 3.6 (Linear GCNN)**.** In the linear case, where $r = 1$, we have

$$\Phi_\theta = x * \varphi_\theta = x * \theta_1 * \ldots * \theta_L.$$

Note then that the coefficients of the polynomial $\Phi_\theta(e) \in \mathrm{Sym}_{\mathbb{K}}(x, 1)$ determine completely the filter $\varphi_\theta \in \mathbb{K}[G]$, and vice versa. Moreover, it is clear that φ is surjective. Therefore in the linear case, we have $\mathcal{M}_\Phi \cong \mathcal{M}_\varphi = \mathbb{K}[G]$.

We describe here broadly our technique to compute the dimension of the neuromanifold $\mathcal{M}_\Phi$ and the dimension of $\mathcal{M}_\varphi$.

1. We study the rank of the Jacobians of the parametrization maps Φ and φ at general filters. Note that for nonzero scalars $\lambda_l \in \mathbb{K} - \{0\}$, where $1 \leq l \leq L - 1$, and any tuple of filters $\theta = (\theta_1, \ldots, \theta_L) \in \mathbb{K}[G]^L$, we have

$$\varphi_\theta = \varphi_\psi \text{ and } \Phi_\theta = \Phi_\psi, \tag{1}$$

   where $\psi = (\lambda_1\theta_1, \lambda_1^r\lambda_2\theta_2, \ldots, \lambda_{L-2}^r\lambda_{L-1}\theta_{L-1}, \lambda_{L-1}^r\theta_L)$. Therefore the rank of the Jacobians $J_\theta\varphi$ and $J_\theta\Phi$ is bounded up by $nL - L + 1 = n(L-1) + 1$.

2. The vectors that lie in the kernels of $J_\theta\varphi$ and $J_\theta\Phi$ due to Eq (1) are all killed by the projectivization of the domain. Therefore if $J_\theta\tilde{\varphi}$ and $J_\theta\tilde{\Phi}$ are full rank, then the affine Jacobians $J_\theta\varphi$ and $J_\theta\Phi$ have the maximal rank $n(L-1) + 1$.

3. To compute the rank of the Jacobians at general filters θ, we consider the Jacobians as symbolic matrices in terms of the parameters θ and compute their symbolic rank.

4 Main results on neuromanifolds of PGCNN

In this section, we state explicitly our main results about the neuromanifolds of PGCNN; we compute the dimensions $\mathcal{M}_\varphi$ and $\mathcal{M}_\Phi$, in addition to give a description of the general fibers of φ and Φ.

4.1 Dimension of $\mathcal{M}_\varphi$ and $\mathcal{M}_\Phi$

Theorem 4.1. *For activation degree $r \geq 2$, the rational map $\tilde{\varphi}$ is* generally regular, *i.e. its Jacobian is full rank at a general point in its domain. This implies that* $\dim(\mathcal{M}_\varphi) = L(n-1) + 1$, *where $n = |G|$ and L is the number of layers.*

Proof. Let $\theta := (\theta_1, \ldots, \theta_L)$ be L general filters on a group G with n elements. The projectivized map $\tilde{\varphi}$ is given by the following composition of maps, $\tilde{\varphi} = p \circ \varphi \circ f$,

$$
\begin{aligned}
(\mathbb{P}(\mathbb{K}^n))^L &\xrightarrow{f} (\mathbb{K}^n)^L \xrightarrow{\varphi} \mathbb{K}^{n^{r^{L-1}}} \xrightarrow{p} \mathbb{P}\left(\mathbb{K}^{n^{r^{L-1}}}\right) \\
([\theta_1], \ldots, [\theta_L]) &\mapsto \underbrace{\left(\frac{\theta_1}{|\theta_1|}, \ldots, \frac{\theta_L}{|\theta_L|}\right)}_{:=\frac{\theta}{|\theta|}} \mapsto \varphi\left(\frac{\theta}{|\theta|}\right) \mapsto \left[\varphi\left(\frac{\theta}{|\theta|}\right)\right] = [\varphi(\theta)],
\end{aligned}
$$

where $|\cdot|$ denotes the Euclidean norm for $V = \mathbb{K}^n$. Note that even though the Euclidean norm depends on the choice of an absolute value for the field $\mathbb{K}$, which exists for all fields, the proof holds regardless of this choice.

Let $[\theta] := ([\theta_1], \ldots, [\theta_L])$, then $J_{[\theta]}\tilde{\varphi} = J_{\varphi(\theta/|\theta|)}p \circ J_{\theta/|\theta|}\varphi \circ J_{[\theta]}f$. We have three observations:

- $\ker\left(J_{\varphi(\theta/|\theta|)}p\right) = \{k\varphi(\theta/|\theta|) \mid k \in \mathbb{K}\}$.
- $\ker\left(J_{\theta/|\theta|}\varphi\right) \cap \operatorname{Im}(J_{[\theta]}f) = \{0\}$, this is because the basis of $\ker\left(J_{\theta/|\theta|}\varphi\right)$, as shown in Proposition 4.2, consists of perturbations that do not alter the class $[\theta]$.
- By Euler's theorem for homogeneous functions, we have $(J_{\theta/|\theta|}\varphi)(\theta/|\theta|) = k\varphi(\theta/|\theta|)$ for some constant $k \in \mathbb{K}$.

Assume $\dot{\theta} \in \ker\left(J_{[\theta]}\tilde{\varphi}\right)$ then $(J_{\theta/|\theta|}\varphi \circ J_{[\theta]}f)(\dot{\theta}) \in \ker J_{\varphi(\theta/|\theta|)}p$ which implies $(J_{\theta/|\theta|}\varphi \circ J_{[\theta]}f)(\dot{\theta}) = k\varphi(\theta/|\theta|)$. Hence $(J_{[\theta]}f)(\dot{\theta}) = k\theta/|\theta| + \dot{\psi}$ where $\dot{\psi} \in \ker\left(J_{\theta/|\theta|}\varphi\right)$. But $k\theta/|\theta| + \dot{\psi}$ is a perturbation that does not alter the class $[\theta]$ hence $\dot{\theta} = 0$. □

The proof of Theorem 4.1 relies strongly on the next proposition.

Proposition 4.2. *Let L be the number of layers and r be the degree of activation. Let also $\theta = (\theta_1, \ldots, \theta_L)$ denote the tuple of filters that define φ_θ. Then for a general θ, the kernel of the Jacobian $J_\theta\varphi$ has a basis of $L-1$ elements given by*

$$
((\theta_1, -r\theta_2, 0, \ldots, 0), (0, \theta_2, -r\theta_3, 0, \ldots, 0), \ldots, (0, \ldots, 0, \theta_{L-1}, -r\theta_L)).
$$

Proof. We do a proof by induction over the number of layers L. The hardest step in this inductive proof is to show that for a perturbation $\dot{\theta} \in \ker(J_\theta\varphi)$ at a general point $\theta \in \mathbb{K}[G]^L$ the last layer filter θ_L and its corresponding perturbation filter $\dot{\theta}_L$ are related by a scaling, i.e. $\dot{\theta}_L = \lambda\theta_L$ for $\lambda \in \mathbb{K}\backslash\{0\}$. For more details, see Appendix A. □

The next theorem states the same result for the dimension of the neuromanifold $\mathcal{M}_\Phi$.

Theorem 4.3. *For activation degree $r \geq 2$, the rational map $\tilde{\Phi}$ is* generally regular*; that is, its Jacobian has full rank at a general point of its domain. This implies* $\dim(\mathcal{M}_\Phi) = L(n-1)+1$.

Proof. The claim follows directly from Proposition 4.4, in the same manner that Proposition 4.2 implies the general regularity of $\tilde{\varphi}$ in Theorem 4.1. □

Proposition 4.4. *Let L be the number of layers and r the activation degree. Let $\theta = (\theta_1, \ldots, \theta_L)$ denote the tuple of filters defining Φ_θ. Then for a general θ, the kernel of the Jacobian $J_\theta\Phi$ has an $(L-1)$–dimensional basis given by*

$$
\big((\theta_1, -r\theta_2, 0, \ldots, 0),\ (0, \theta_2, -r\theta_3, 0, \ldots, 0),\ \ldots,\ (0, \ldots, 0, \theta_{L-1}, -r\theta_L) \big).
$$

Proof. See Appendix B. □

4.2 General fiber of φ

The fiber of φ at filters $\theta := (\theta_1, \ldots, \theta_L)$ is the set

$$
\varphi^{-1}(\varphi_\theta) := \{\psi \in \mathbb{K}[G]^L \mid \varphi_\psi = \varphi_\theta\}.
$$

In this subsection, the shape of the general fiber of φ is described; however, before we give this main result, we need to borrow a lemma about symmetric tensor ranks from [CGLM08].

Lemma 4.5 (Lemma 5.1 [CGLM08]). *Let $y_1, \ldots, y_n \in \mathbb{K}^d$ be linearly independent. Then the symmetric tensor*

$$A := \sum_{i=1}^{n} y_i^{\otimes r}$$

has a symmetric rank $\text{rank}_S(A) = n$.

Theorem 4.6. *For general filters $\theta := (\theta_1, \ldots, \theta_L)$, we have $\tilde{\varphi}_\theta = \tilde{\varphi}_\psi$ if and only if*

$$\psi = (\theta_1 * g_1,\ g_1^{-1} * \theta_2 * g_2,\ \ldots,\ g_{L-2}^{-1} * \theta_{L-1} * g_{L-1},\ g_{L-1}^{-1} * \theta_L),$$

where $g_i \in G$ for $1 \leq i \leq L-1$, up to rescaling each filter.

Proof. We do a proof by induction over the number of layers L. The base case is trivial and the fiber is a , up to rescaling the only filter.

Assume the proposition holds for all numbers of layers less than or equal to $L-1$. Let θ' and ψ' denote the first $L-1$ filters in θ and ψ, and consider the case for L to get

$$\varphi_\theta = \varphi_{\theta'}^{\otimes r} * \theta_L = \lambda \varphi_{\psi'}^{\otimes r} * \psi_L = \lambda \varphi_\psi,$$

where $\lambda \in \mathbb{K} \backslash \{0\}$. Since θ is general, by Proposition 2.20 we get that $\varphi_\theta \in \mathbb{K}[G^{r^{L-1}}]$ has an inverse filter φ_θ^{-1} and, moreover, $\psi_L \in \mathbb{K}[G^{r^{L-1}}]$ has an inverse given by $\psi_L^{-1} = \varphi_\theta^{-1} * \lambda \varphi_{\psi'}^{\otimes r}$. Convolute both sides above by ψ_L^{-1} to get

$$\varphi_{\theta'}^{\otimes r} * \theta_L * \psi_L^{-1} = \lambda \varphi_{\psi'}^{\otimes r}.$$

Define $\mu := \theta_L * \psi_L^{-1}$ and substitute in the above equation to get

$$\varphi_{\theta'}^{\otimes r} * \mu = \lambda \varphi_{\psi'}^{\otimes r}.$$

Write $\mu = \sum_{g \in G} \mu(g) g$ and use Proposition 2.12 to distribute convolution over the Kronecker product to get

$$\sum_{g \in G} \mu(g) \varphi_{\theta'} * g \otimes \ldots \otimes \varphi_{\theta'} * g = \lambda \varphi_{\psi'} \otimes \ldots \otimes \varphi_{\psi'}.$$

Notice that for any $g \in G$, $\varphi_{\theta'} * g$ is a vector in $\mathbb{K}^{n^{r^{L-2}}}$, and more importantly, it is a column in the matrix $\text{Mat}_{\varphi_{\theta'}}$. For general filters $\theta_1, \ldots, \theta_{L-1}$, Proposition 2.20 shows the determinant $\det(\text{Mat}_{\varphi_{\theta'}}) \neq 0$ and so the vectors $\{\varphi_{\theta'} * g \mid g \in G\}$ are linearly independent. Therefore by Lemma 4.5, the above equation holds if and only if $\mu \in G$ up to a scaling, which implies $\theta_L = g * \psi_L$ up to a rescaling. The rest follows by induction. □

4.3 General fiber of Φ

In this subsection, we discuss the general fiber of the parametrization map Φ. We develop a conjecture similar in nature to Lemma 4.5, and based on this conjecture, we present the shape of the general fiber.

Denote by $VP_{n,k,d}$ the variety of kth power of degree d homogeneous polynomials in n variables x [AC14]. Namely,

$$VP_{n,k,d} := \{p \in \text{Sym}_{\mathbb{K}}(x, kd) \mid \exists q \in \text{Sym}_{\mathbb{K}}(x, d),\ q^k = p\}.$$

Choose m polynomials in $p_1, \ldots, p_m \in VP_{n,k,d}$, we call their linear span $< p_1, \ldots, p_m >$ in $\text{Sym}_{\mathbb{K}}(x, kd)$ by an *m-secant* line of $VP_{n,k,d}$.

Conjecture 4.7. *Let $G = \{g_1, \ldots, g_n\}$ as a set, and let the activation degree $r \geq 2$. For general filters $\theta = (\theta_1, \ldots, \theta_L)$, where $L \geq 1$ we have*

$$< \sigma_r(\Phi_\theta)(g_1), \ldots, \sigma_r(\Phi_\theta)(g_n) > \cap\ VP_{n,r,r^{L-1}} = \{\sigma_r(\Phi_\theta)(g_1), \ldots, \sigma_r(\Phi_\theta)(g_n)\}.$$

This conjecture describes the intersection of the variety $VP_{n,r,r^{L-1}}$ with its n-secant lines. Note that when the n-secant line is general, then the conclusion of the conjecture is trivial. However, in the conjecture, we apply constraints on the considered secant lines. For example, the first polynomial $\sigma_r(\Phi_\theta)(g_1)$ determines, by equivariance, the rest of the $n-1$ polynomials. The problem here is similar in nature to the invertibility of G-circulent matrices that we dealt with in §2. While a general square matrix is clearly invertible, we needed to do some extra work to show that a G-circulent matrix is also generally invertible.

Theorem 4.8 (Conditional on Conjecture 4.7)**.** *For general filters $\theta := (\theta_1, \ldots, \theta_L)$, we have $\tilde{\Phi}_\theta = \tilde{\Phi}_\psi$ if and only if $\psi := (\theta_1 * g_1,\ g_1^{-1} * \theta_2 * g_2,\ \ldots,\ g_{L-2}^{-1} * \theta_{L-1} * g_{L-1},\ g_{L-1}^{-1} * \theta_L)$, where $g_i \in G$ for $1 \leq i \leq L-1$, up to rescaling each filter.*

Proof. The proof is similar to Theorem 4.6 and proceeds by induction over the number of layers L. The base case is trivial and the fiber is a singleton, up to rescaling.

Assume the theorem holds for all numbers of layers less than or equal to $L-1$. Let θ' and ψ' denote the first $L-1$ filters in θ and ψ, and consider the case for L to get

$$\Phi_\theta = \sigma_r(\Phi_{\theta'}) * \theta_L = \lambda \sigma_r(\Phi_{\psi'}) * \psi_L = \lambda \Phi_\psi, \tag{2}$$

where $\lambda \in \mathbb{K} - \{0\}$. Since θ is general, we get that Φ_θ has an inverse filter and, moreover, ψ_L has an inverse. Namely, evaluated at a general signal x, Φ_θ becomes a $\mathbb{K}$ filter over G that is invertible and $\psi_L^{-1} = \Phi_\theta^{-1} * \lambda \sigma_r(\Phi_{\psi'})$. Convolute both sides of equation (2) by ψ_L^{-1} to get

$$\sigma_r(\Phi_{\theta'}) * \theta_L * \psi_L^{-1} = \lambda \sigma_r(\Phi_{\psi'}).$$

Define $\mu := \theta_L * \psi_L^{-1}$ and substitute in the last equation to get

$$\sigma_r(\Phi_{\theta'}) * \mu = \sigma_r(\Phi_{\psi'}).$$

Then evaluate at $e \in G$ to get

$$\sum_{g \in G} \sigma_r(\Phi_{\theta'})(g) \mu(g^{-1}) = \lambda \sigma_r(\Phi_{\psi'})(e).$$

The last equation says that $\lambda \sigma_r(\Phi_{\psi'})(e)$ lies in the n-secant line generated by $\{\sigma_r(\Phi_{\theta'})(g)\}_{g \in G}$. So based on Conjecture 4.7, we get $\lambda \sigma_r(\Phi_{\psi'})(e) = \sigma_r(\Phi_{\theta'})(g)$ for some $g \in G$. As a result, we have $\mu = \lambda g$ for some $g \in G$, or equivalently $\theta_L = \lambda g * \psi_L$. The rest follows by induction.

□

5 Conclusion and Outlook

In this work, we presented a mathematical framework for polynomial group convolutional networks using graded group algebras. Under this framework, we computed the dimension of the neuromanifold for a PGCNN where the group G is an arbitrary finite group. We showed that the dimension does not depend on the group structure but only on the size of the group. We also present a conjecture on the general fiber of these networks.

For future work, we aim to prove Conjecture 4.7 by studying explicitly the defining polynomials of the variety $VP_{n,k,d}$ as stated in [AC14]. Furthermore, we believe that techniques used in [SMK24] to compute the degree and the generic Euclidean degree for CNN can be generalized and used here. In the CNN case, considered in [SMK24], the parametrization map Φ factors through Segre-Veronese embedding via a projective linear map. Since the degree of a variety is invariant under projective linear morphism, they get that the degree of the Segre-Veronese variety naturally translates into the degree of the neurovariety of a CNN. In contrast, in the case of PGCNN, the parametrization map Φ factors through the Segre-Veronese embedding via a rational map with a nontrivial kernel rather than a projective linear morphism. Therefore, in the case of PGCNN, we expect the Segre-Veronese factorization of the map Φ to give an upper bound for both the degree and the generic Euclidean degree, rather than determining them completely.

Acknowledgements

This work was in part supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. The computations were partly enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by Vetenskapsrådet through Grant Agreement.

We also thank Konstantin Usevich and Giorgio Ottaviani for discussions and suggestions.

References

[AC14] Abdelmalek Abdesselam and Jaydeep Chipalkatti. On hilbert covariants. *Canadian Journal of Mathematics*, 66(1):3–30, February 2014.

[BBCV21] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges, 2021.

[CGLM08] Pierre Comon, Gene Golub, Lek-Heng Lim, and Bernard Mourrain. Symmetric tensors and symmetric tensor rank, 2008.

[CGW19] Taco S. Cohen, Mario Geiger, and Maurice Weiler. *A general theory of equivariant CNNs on homogeneous spaces*. Curran Associates Inc., Red Hook, NY, USA, 2019.

[CW16] Taco Cohen and Max Welling. Group equivariant convolutional networks. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2990–2999, New York, New York, USA, 20–22 Jun 2016. PMLR.

[DWZ+18] Simon S Du, Yining Wang, Xiyu Zhai, Sivaraman Balakrishnan, Russ R Salakhutdinov, and Aarti Singh. How many samples are needed to estimate a convolutional neural network? In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

[FRWY25] Bella Finkel, Jose Israel Rodriguez, Chenxi Wu, and Thomas Yahl. Activation degree thresholds and expressiveness of polynomial neural networks, 2025.

[KMMT22] Kathlén Kohn, Thomas Merkh, Guido Montúfar, and Matthew Trager. Geometry of linear convolutional networks, 2022.

[KMST24] Kathlén Kohn, Guido Montúfar, Vahid Shahverdi, and Matthew Trager. Function space and critical points of linear convolutional networks, 2024.

[KTB19] Joe Kileel, Matthew Trager, and Joan Bruna. On the expressive power of deep polynomial neural networks. *CoRR*, abs/1905.12207, 2019.

[MCTH18] Dhagash Mehta, Tianran Chen, Tingting Tang, and Jonathan D. Hauenstein. The loss surface of deep linear networks viewed through the algebraic geometry lens, 2018.

[MM25] A. Massarenti and M. Mella. The alexander-hirschowitz theorem for neurovarieties, 2025.

[MSM+25] Giovanni Luca Marchetti, Vahid Shahverdi, Stefano Mereta, Matthew Trager, and Kathlén Kohn. Algebra unveils deep learning – an invitation to neuroalgebraic geometry, 2025.

[Sha24] Vahid Shahverdi. Algebraic complexity and neurovariety of linear convolutional networks, 2024.

[SMK24] Vahid Shahverdi, Giovanni Luca Marchetti, and Kathlén Kohn. On the geometry and optimization of polynomial convolutional networks, 2024.

[TKB20] Matthew Trager, Kathlén Kohn, and Joan Bruna. Pure and spurious critical points: a geometric study of linear networks, 2020.

[UBDC25] Konstantin Usevich, Ricardo Borsoi, Clara Dérand, and Marianne Clausel. Identifiability of deep polynomial neural networks, 2025.

A Proof of Proposition 4.2

Before we state the proof of Proposition 4.2, we need to set some notation that applies only in this appendix. We also need to prove two lemmata, one easy and one hard.

Notation.

- We introduce the following notation

$$[\breve{x}_1 \diamond x_2 \diamond \ldots \diamond x_n] := \breve{x}_1 \diamond x_2 \diamond \ldots \diamond x_n \; + \; x_1 \diamond \breve{x}_2 \diamond \ldots \diamond x_n \; + \ldots + \; x_1 \diamond \ldots \diamond \breve{x}_n,$$

  where $\breve{}$ and $\diamond$ denote arbitrary operations.

- Let $g \in G^r$ and $h \in G$ we define $g \cdot h := g \cdot h^r$.

Lemma A.1. *Let θ and ψ be G-filters, then*

1.

$$J_{\theta,\psi}(\theta * \psi)(\dot{\theta}, \dot{\psi}) = \dot{\theta} * \psi + \theta * \dot{\psi} = [\dot{\theta} * \psi].$$

2.

$$\begin{aligned} J_\theta(\theta^{\otimes r})(\dot{\theta}) &= \dot{\theta} \otimes \theta \otimes \ldots \otimes \theta + \theta \otimes \dot{\theta} \otimes \ldots \otimes \theta + \ldots + \theta \otimes \ldots \otimes \theta \otimes \dot{\theta} \\ &= \dot{\theta} \otimes \theta^{\otimes(r-1)} + \theta \otimes \dot{\theta} \otimes \theta^{\otimes(r-2)} + \ldots + \theta^{\otimes(r-1)} \otimes \dot{\theta} \\ &= [\dot{\theta} \otimes \theta \otimes \ldots \otimes \theta]. \end{aligned}$$

3. Let $\theta = (\theta_1, \ldots, \theta_L)$, $\dot{\theta} = (\dot{\theta}_1, \ldots, \dot{\theta}_L)$. Moreover, let θ' and $\dot{\theta}'$ be the tuples containing the first $L-1$ filters in θ and $\dot{\theta}$, respectively. Since $\varphi_\theta = \varphi_{\theta'}^{\otimes r} * \theta_L|^{G^{r^{L-1}}}$, we get

$$J_\theta \varphi(\dot{\theta}) = [J_{\theta'} \varphi(\dot{\theta}') \otimes \underbrace{\varphi_{\theta'} \otimes \ldots \otimes \varphi_{\theta'}}_{=\varphi_{\theta'}^{\otimes(r-1)}}] * \theta_L + \varphi_{\theta'}^{\otimes r} * \dot{\theta}_L.$$

Proof. 1.

$$J_{\theta,\psi}(\theta * \psi)(\dot{\theta}, \dot{\psi}) = J_{\theta,\psi}\left(\left(\sum_h \theta(g_1 \cdot h^{-1})\psi(h), \ldots, \sum_h \theta(g_n \cdot h^{-1})\psi(h)\right)\right)(\dot{\theta}, \dot{\psi}).$$

Since $J_{\theta,\psi}\left(\sum_h \theta(g_i \cdot h^{-1})\psi(h)\right)(\dot{\theta}, \dot{\psi}) = \sum_h \dot{\theta}(g_i \cdot h^{-1})\psi(h) + \sum_h \theta(g_i \cdot h^{-1})\dot{\psi}(h)$, we get

$$J_{\theta,\psi}(\theta * \psi)(\dot{\theta}, \dot{\psi}) = \dot{\theta} * \psi + \theta * \dot{\psi}.$$

2. We prove this for $r = 2$, the general case follows by induction.

   We have $\theta \otimes \theta = (\theta_1^2, \theta_1\theta_2, \ldots, \theta_1\theta_n, \theta_2\theta_1, \ldots, \theta_2\theta_n, \ldots, \theta_n\theta_1, \ldots, \theta_n^2)$, hence $J_\theta(\theta \otimes \theta)$ is a $n^2 \times n$ matrix where the $((i-1)n+j)$th row corresponding to the function $\theta_i\theta_j$ will have θ_j at the ith column, θ_i a the jth column, and zero everywhere else. Therefore, the $((i-1)n+j)$th row in $J_\theta(\theta \otimes \theta)(\dot{\theta})$ is equal to $\theta_i\dot{\theta}_j + \dot{\theta}_i\theta_j$.

3. Direct application of the first rule, then the second rule.

□

Lemma A.2. *Let $\theta = (\theta_1, \ldots, \theta_{L-1}, e) \in \mathbb{K}[G]^L$ where θ_i is a general filter for $1 \leq i < L$ and $e \in G \subset \mathbb{K}[G]$ is the identity element. If $\dot{\theta} = (\dot{\theta}_1, \ldots, \dot{\theta}_L) \in \ker J_\theta\varphi$ then $\dot{\theta}_L = \lambda e$ for some $\lambda \in \mathbb{K}$.*

Proof. Write $\theta' = (\theta_1, \ldots, \theta_{L-1})$ and similarly $\dot{\theta}' = (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1})$. For brevity, write,

$$J := J_{\theta'}\varphi(\dot{\theta}').$$

By Lemma A.1 (the Jacobian computation rules), $\dot{\theta} \in \ker\big(J_\theta\varphi\big)$ if and only if

$$\left[J \otimes \varphi_{\theta'}^{\otimes(r-1)}\right] \; = \; -\,\varphi_{\theta'}^{\otimes r} * \dot{\theta}_L \tag{3}$$

Evaluate (3) at tuples of the special form $\tilde{g}_i = (g_i, \ldots, g_i) \in G^{r^{L-1}}$ (where $g_i \in G^{r^{L-2}}$). This yields, for each i,

$$r\,(\varphi_{\theta'}(g_i))^{r-1} J(g_i) \; = \; \sum_{h \in G} (\varphi_{\theta'}(g_i h^{-1}))^r \dot{\theta}_L(h).$$

Hence

$$J(g_i) \; = \; \frac{1}{r}\,\varphi_{\theta'}(g_i)\,\dot{\theta}_L(e) \; + \; \frac{1}{r} \sum_{h \in G \setminus \{e\}} \frac{(\varphi_{\theta'}(g_i h^{-1}))^r}{(\varphi_{\theta'}(g_i))^{r-1}}\,\dot{\theta}_L(h). \tag{4}$$

Now evaluate (3) at $\tilde{g} = (g_{i_1}, \ldots, g_{i_r})$ to get,

$$\sum_{k=1}^{r} J(g_{i_k}) \prod_{j \neq k} \varphi_{\theta'}(g_{i_j}) = \sum_{h \in G} \varphi_{\theta'}^{\otimes r}(\tilde{g}h^{-1})\dot{\theta}_L(h).$$

Substituting (4) in the above equation gives a linear system in the unknowns $\{\dot{\theta}_L(h)\}_{h \neq e}$ with coefficient matrix C whose entries are symbolic expressions in the values $\varphi_{\theta'}(g_i)$. Denote these coefficients by

$$c_{\tilde{g},h} = \frac{1}{r} \sum_{k=1}^{r} \left(\frac{(\varphi_{\theta'}(g_{i_k}h^{-1}))^r \prod_{j \neq k} \varphi_{\theta'}(g_{i_j})}{(\varphi_{\theta'}(g_{i_k}))^{r-1}} \right) - \varphi_{\theta'}^{\otimes r}(\tilde{g}h^{-1}).$$

Observe $c_{\tilde{g},e} = 0$ for every $\tilde{g}$ that is why we exclude the variable $\dot{\theta}_L(e)$ from the unknowns. The matrix C has $|G^{r^{L-1}}| = n^{r^{L-1}}$ rows and $n-1$ columns, so C is full column rank if and only if some $(n-1) \times (n-1)$ minor is nonzero as a symbolic polynomial in the variables θ'.

Choose the $(n-1)$ rows corresponding to

$$\tilde{g}^{\,i} := (g_1, g_i, \ldots, g_i) \qquad (2 \leq i \leq n),$$

where $g_j = (h_j, \ldots, h_j) \in G^{r^{L-2}}$, $h_1 = e$ and $h_j \in G$. For this choice, one checks that the diagonal entries of the resulting $(n-1)$-minor contain a factor $(\varphi_{\theta'}(g_1))^{r+1}$ which does not appear elsewhere in that minor. Namely,

$$c_{\tilde{g}^i,h_j} = \frac{1}{r} \frac{(\varphi_{\theta'}(g_i))^{r-1}}{(\varphi_{\theta'}(g_1))^{r-1}} \varphi_{\theta'}((g_1 h_j^{-1}))^r + \frac{r-1}{r} \frac{(\varphi_{\theta'}(g_i h^{-j}))^r}{\varphi_{\theta'}(g_i)} \varphi_{\theta'}(g_1) - \varphi_{\theta'}^{\otimes r}(\tilde{g}^i h_j^{-1}).$$

and

$$c_{\tilde{g}^i,h_i} = \frac{1}{r} \frac{(\varphi_{\theta'}(g_i))^{r-1}}{(\varphi_{\theta'}(g_1))^{r-1}} \varphi_{\theta'}((g_1 h_i^{-1}))^r + \frac{r-1}{r} \frac{(\varphi_{\theta'}(g_1))^{r+1}}{\varphi_{\theta'}(g_i)} - \varphi_{\theta'}^{\otimes r}(\tilde{g}^i h_i^{-1}).$$

Since $\varphi_{\theta'}(g_1)$ has monomials in the parameters of θ' that do not occur in $\varphi_{\theta'}(g_i)$ for $2 \geq i \leq n$, then this symbolic minor is nonvanishing for general θ', so C is symbolically full rank. Therefore the only solution is $\dot{\theta}_L(h) = 0$ for all $h \neq e$. □

We are finally ready to give a proof of Proposition 4.2.

Proof of Proposition 4.2. Let the perturbations be $\dot{\theta} = (\dot{\theta}_1, \ldots, \dot{\theta}_L)$ and write $\theta' = (\theta_1, \ldots, \theta_{L-1})$ and similarly $\dot{\theta}' = (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1})$. We prove the claim by induction on L.

Base case $L = 1$. If $L = 1$ then $\varphi_\theta = \theta_1$ and

$$J_\theta \varphi(\dot{\theta}) = J_{\theta_1}(\theta_1)(\dot{\theta}_1) = \dot{\theta}_1,$$

so $J_\theta \varphi$ is the identity and hence full rank. The kernel is trivial, which matches the statement for $L = 1$.

Inductive step. Assume the proposition holds for $L-1$ layers; we prove it for L layers. For brevity, write,

$$J := J_{\theta'} \varphi(\dot{\theta}').$$

By Lemma A.1 (the Jacobian composition / convolution rule), $\dot{\theta} \in \ker\big(J_\theta \varphi\big)$ if and only if

$$\big[J \otimes \varphi_{\theta'}^{\otimes(r-1)}\big] * \theta_L \;=\; -\varphi_{\theta'}^{\otimes r} * \dot{\theta}_L. \tag{5}$$

If $\dot{\theta}_L = 0$ then $J = 0$ the result follows by the inductive step. Assume $\dot{\theta}_L \neq 0$, convolute both sides of (5) with the inverse of θ_L, which exists because θ_L is general, and let $\dot{\tilde{\theta}}_L := \dot{\theta}_L * \theta_L^{-1}$ to get

$$\big[J \otimes \varphi_{\theta'}^{\otimes(r-1)}\big] \;=\; -\varphi_{\theta'}^{\otimes r} * \dot{\tilde{\theta}}_L. \tag{6}$$

Since θ' is general and $\dot{\theta}_L \neq 0$, then by Lemma A.2, we have $\dot{\tilde{\theta}}_L = \lambda e$ for some nonzero $\lambda \in \mathbb{K}$. This implies $\dot{\theta}_L = \lambda \theta_L$, and Therefore, any $\dot{\theta} \in \ker J_\theta \varphi$ must have the form

$$\dot{\theta} = (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1},\ \lambda \theta_L).$$

Observe that the vector

$$(0, \ldots, 0,\ -\tfrac{\lambda}{r}\theta_{L-1},\ \lambda \theta_L)$$

lies in the kernel as well (it is a direct check using the Jacobian composition rule). Subtracting this vector from $\dot{\theta}$ yields

$$(\dot{\theta}_1, \ldots, \dot{\theta}_{L-1} - (-\tfrac{\lambda}{r}\theta_{L-1}),\ 0) \ = \ (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1} + \tfrac{\lambda}{r}\theta_{L-1},\ 0) \in \ker J_\theta \varphi.$$

By Eq. (5), the vector

$$(\dot{\theta}_1, \ldots, \dot{\theta}_{L-1} + \tfrac{\lambda}{r}\theta_{L-1}) \in \ker J_{\theta'} \varphi,$$

and by the inductive hypothesis, it can be written as a linear combination of

$$\big((\theta_1, -r\theta_2, 0, \ldots, 0),\ (0, \theta_2, -r\theta_3, 0, \ldots, 0),\ \ldots,\ (0, \ldots, 0, \theta_{L-2}, -r\theta_{L-1})\big).$$

Adding back the previously subtracted vector $(0, \ldots, 0, -\frac{\lambda}{r}\theta_{L-1}, \lambda\theta_L)$ shows that $\dot{\theta}$ is a linear combination of the $L-1$ vectors stated in the proposition. This completes the induction and the proof. □

B Proof of Proposition 4.4

In this appendix we provide a complete proof of Proposition 4.4. However, before we do that, we will need to introduce new notation and two lemmata.

Notation. Recall that a polynomial G-convolutional neural network of depth L, and activation r is the map

$$\begin{aligned} \Phi : \mathbb{K}[G]^L &\to \mathrm{Sym}_{\mathbb{K}}(n, r^{L-1})[G] \\ (\theta_1, \ldots, \theta_L) &\mapsto \sigma_r\Big(\cdots \sigma_r\big(\sigma_r(x * \theta_1) * \theta_2\big) \cdots * \theta_{L-1}\Big) * \theta_L. \end{aligned}$$

Equivalently, the map Φ can be defined recursively by the formula

$$\Phi(\theta_1, \ldots, \theta_L) = \sigma_r\big(\Phi(\theta_1, \ldots, \theta_{L-1})\big) * \theta_L.$$

Recall also from Section 3 that the map Φ can also be defined in terms of the map φ by the formula

$$\Phi(\theta_1, \ldots, \theta_L) = \big(x^{r^{L-1}} * \varphi(\theta_1, \ldots, \theta_L)\big)\big|_G. \tag{7}$$

For $\theta := (\theta_1, \ldots, \theta_L) \in \mathbb{K}[G]^L$, define the filter Φ_θ to be the one obtained by evaluating Φ at θ i.e.

$$\begin{aligned} \Phi_\theta : G &\to \mathrm{Sym}_{\mathbb{K}}(n, r^{L-1}) \\ g &\mapsto \Phi(\theta)(g). \end{aligned}$$

Each monomial in $\Phi_\theta(g) \in \mathrm{Sym}_{\mathbb{K}}(n, r^{L-1})$ is indexed by an unordered tuple

$$[g_1, \ldots, g_{r^{L-1}}] \in G^{r^{L-1}},$$

or, equivalently, by multiplicities

$$[r_1 g_1, \ldots, r_j g_j], \qquad r_1 + \cdots + r_j = r^{L-1}.$$

We denote the coefficient of this monomial in $\Phi_\theta(g)$ by

$$\Phi_\theta(g)[r_1 g_1, \ldots, r_j g_j],$$

or simply $\Phi_\theta[r_1 g_1, \ldots, r_j g_j]$ when the group element g is clear from context.

Lemma B.1. *Let $\theta = (\theta_1, \ldots, \theta_L)$ and $\dot{\theta} = (\dot{\theta}_1, \ldots, \dot{\theta}_L)$, and set*

$$\theta' = (\theta_1, \ldots, \theta_{L-1}), \qquad \dot{\theta}' = (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1}).$$

Then:

1. *For any index $[g_1, \ldots, g_{r^{L-1}}]$ we have*

$$\Phi_\theta(g)[g_1, \ldots, g_{r^{L-1}}] = \sum_{\delta \in S_{r^{L-1}}} \varphi_\theta(g_{\delta(1)}^{-1} g, \ldots, g_{\delta(r^{L-1})}^{-1} g) = \Phi_\theta(e)[g^{-1} g_1, \ldots, g^{-1} g_{r^{L-1}}], \tag{8}$$

where S_m denotes the permutation group on m elements.

2. *As proved in [SMK24],*

$$J_\theta \Phi(\dot\theta) = r\big(J_{\theta'}\Phi(\dot\theta') \odot \sigma_{r-1}(\Phi_{\theta'})\big) * \theta_L \;+\; \sigma_r(\Phi_{\theta'}) * \dot\theta_L.$$

The Jacobian of Φ_θ is obtained by computing the Jacobian of each polynomial coefficient of $\Phi_\theta(g)$ for all $g \in G$.

Proof. Using (7), for any $g \in G$,

$$\Phi_\theta(g) = \sum_{h \in G^{r^{L-1}}} x^{r^{L-1}}(h)\, \varphi_\theta(h^{-1}g).$$

Since $x^{r^{L-1}}(h_1) = x^{r^{L-1}}(h_2)$ whenever h_1, h_2 differ by a permutation, the formula (8) follows immediately. □

In other words, the first part of Lemma B.1 states that the coefficients of the polynomial $\Phi_\theta(g)$ are completely determined by the coefficients of the polynomial $\Phi_\theta(e)$ at the group identity $e \in G$.

Lemma B.2. *Let $\theta = (\theta_1, \dots, \theta_{L-1}, e) \in \mathbb{K}[G]^L$ where θ_i is a general filter for $1 \le i < L$ and $e \in G \subset \mathbb{K}[G]$ is the identity element. If $\dot\theta = (\dot\theta_1, \dots, \dot\theta_L) \in \ker J_\theta\Phi$ then $\dot\theta_L = \lambda e$ for some $\lambda \in \mathbb{K}$.*

Proof. Let $G = \{g_1 = e, g_2, \dots, g_n\}$, $\theta' = (\theta_1, \dots, \theta_{L-1})$ and $\dot\theta' = (\dot\theta_1, \dots, \dot\theta_{L-1})$. For brevity set

$$J := J_{\theta'}\Phi(\dot\theta').$$

Lemma B.1 implies that $\dot\theta \in \ker J_\theta\Phi$ if and only if

$$rJ \odot \sigma_{r-1}(\Phi_{\theta'}) = -\,\sigma_r(\Phi_{\theta'}) * \dot\theta_L. \tag{9}$$

Evaluate the filters on both sides in Eq. (9) at $g = e$ to get

$$(rJ \odot \sigma_{r-1}(\Phi_{\theta'}))(e) = -\sum_{h \in G} \sigma_r(\Phi_{\theta'})(h)\dot\theta_L(h^{-1}). \tag{10}$$

Equation (10) is an equality of two homogeneous polynomials in $\mathrm{Sym}_{\mathbb{K}}(x, r^{L-1})$. Therefore for an index $[r_1g_1, \dots, r_ng_n]$ with $r_1 + \cdots + r_n = r^{L-1}$, equation (10) gives the equation

$$(rJ \odot \sigma_{r-1}(\Phi_{\theta'}))(e)[r_1g_1, \dots, r_ng_n] = -\sum_{h \in G} \sigma_r(\Phi_{\theta'})(h)[r_1g_1, \dots, r_ng_n]\dot\theta_L(h^{-1}),$$

which by Proposition B.1 is equivalent to

$$(rJ \odot \sigma_{r-1}(\Phi_{\theta'}))(e)[r_1g_1, \dots, r_ng_n] = -\sum_{h \in G} \sigma_r(\Phi_{\theta'})(e)[r_1hg_1, \dots, r_nhg_n]\dot\theta_L(h).$$

Since the above equation is a relation between the polynomials at only e, we remove e from the notation and write

$$(rJ \odot \sigma_{r-1}(\Phi_{\theta'}))[r_1g_1, \dots, r_ng_n] = -\sum_{h \in G} \sigma_r(\Phi_{\theta'})[r_1hg_1, \dots, r_nhg_n]\dot\theta_L(h). \tag{$*$}$$

These equations ($*$) form a system of equations in the unknowns $\dot\theta$; one equation for each multindex $[r_1g_1, \dots, r_ng_n]$. Note that on the one hand an unknown appears in the homogeneous polynomial J if and only if it comes from the first $L-1$ filters in $\dot\theta$. On the other hand, the claim of this lemma concerns only the unknowns $\dot\theta_L$. Therefore our goal is to eliminate the unknown homogeneous polynomial J from the system ($*$). To achieve this, we need to express the coefficient of J in terms of $\dot\theta_L$. We do this step by step as follows.

First coefficients. For the index $[r^{L-1}g_i]$, Eq ($*$) becomes

$$r(J \odot \sigma_{r-1}(\Phi_{\theta'}))[r^{L-1}g_i] = -\sum_{h \in G} \sigma_r(\Phi_{\theta'})[r^{L-1}hg_i]\,\dot\theta_L(h).$$

Unfolding the Hadamard product on the left hand side gives

$$r(J \odot \sigma_{r-1}(\Phi_{\theta'}))[r^{L-1}g_i] = rJ[r^{L-2}g_i]\,\sigma_{r-1}(\Phi_{\theta'})[(r-1)r^{L-2}g_i].$$

So we obtain

$$J[r^{L-2}g_i] = -\sum_{h\in G} \frac{\sigma_r(\Phi_{\theta'})[r^{L-1}hg_i]}{r\,\sigma_{r-1}(\Phi_{\theta'})[(r-1)r^{L-2}g_i]}\dot\theta_L(h). \tag{$\dagger$}$$

Next coefficients. For the multiindex $[g_1, (r^{L-1}-1)g_i]$,

$$r(J \odot \sigma_{r-1}(\Phi_{\theta'}))[g_1, (r^{L-1}-1)g_i] = -\sum_{h\in G} \sigma_r(\Phi_{\theta'})[hg_1, (r^{L-1}-1)hg_i)]\,\dot\theta_L(h).$$

The left side expands as

$$rJ[g_1, (r^{L-2}-1)g_i]\,\sigma_{r-1}(\Phi_{\theta'})[(r-1)r^{L-2}g_i] + rJ[r^{L-2}g_i]\,\sigma_{r-1}(\Phi_{\theta'})[g_1, ((r-1)r^{L-2}-1)g_i],$$

so together with ($\dagger$) this expresses $J[g_1, (r^{L-2}-1)g_i]$ linearly in $\dot\theta_L$.

Iterating. Define for $0 \le m \le r^{L-2}$

$$O^i_m = \sum_{h\in G} \frac{\sigma_r(\Phi_{\theta'})[mhg_1, (r^{L-1}-m)hg_i]}{r\,\sigma_{r-1}(\Phi_{\theta'})[(r-1)r^{L-2}g_i]}\dot\theta_L(h),$$

and

$$D^i_k = \sigma_{r-1}(\Phi_{\theta'})[kg_1, ((r-1)r^{L-2}-k)g_i].$$

Define O_m to be the square $n\times n$ matrix where its rows are given by O^i_m. Namely, the entry (i,h) is given by

$$\frac{\sigma_r(\Phi_{\theta'})[mhg_1, (r^{L-1}-m)hg_i]}{r\,\sigma_{r-1}(\Phi_{\theta'})[(r-1)r^{L-2}g_i]}.$$

For $0 \le m < k \le r^{L-2}$ set

$$C^i_{m,k} = \sigma_{r-1}(\Phi_{\theta'})[(k-m)g_1, ((r-1)r^{L-2}+m-k)g_i] = D^i_{k-m},$$

and

$$C^{m,k}_i = \sum_{\{\alpha_1=m<\cdots<\alpha_f=k\}} \prod_{j=1}^{f-1} C^i_{\alpha_j,\alpha_{j+1}}, \qquad C^{k,k}_i = 1.$$

Induction on r_1 yields, for $0 \le r_1 \le r^{L-2}-1$ and $1 < i \le n$:

$$J[r_1g_1, (r^{L-2}-r_1)g_i] = \sum_{m=0}^{r_1} O^i_m\, C^{m,r_1}_i. \tag{$\clubsuit$}$$

Constructing the linear system only in $\dot\theta_L$. Using ($\clubsuit$) and ($\dagger$), for each $i \ne 1$ the coefficient equation ($*$) corresponding to $[r^{L-2}g_1, (r-1)r^{L-2}g_i]$ gives

$$r\sum_{r_1=0}^{r^{L-2}} J[r_1g_1, (r^{L-2}-r_1)g_i]\, D^i_{r^{L-2}-r_1} + rO^i_{r^{L-2}}D^i_0 = 0.$$

This is linear in $\dot\theta_L$, since each $J[\cdot]$ and O^i_m is linear in $\dot\theta_L$.

These $n-1$ equations (indexed by $i \ne 1$) form a $(n-1)\times n$ symbolic matrix in the parameters θ'. The entry in row i and column $h = g_i^{-1}$ contains the term

$$\big(\Phi_{\theta'}[r^{L-2}g_1])\big)^{r+1}\big(\Phi_{\theta'}[r^{L-2}g_i]\big)^{r-1},$$

which is a polynomial in the parameters θ' (the term appears in the sum where $r_1 = 0$). Excluding the entry at the column $h = e$, no other entry in this row contains a term that has the factor $\big(\Phi_{\theta'}[r^{L-2}g_1])\big)^{r+1}$.

Hence the submatrix omitting the column $h = e$ has a symbolically nonvanishing determinant, so the kernel is one–dimensional and spanned by $(1, 0, \ldots, 0)$. Therefore $\dot\theta_L(h) = 0$ for $h \ne e$. This completes the proof of the lemma. □

We are finally ready to give the proof of Proposition 4.4

Proof of Proposition 4.4. We write perturbations as $\dot{\theta} = (\dot{\theta}_1, \ldots, \dot{\theta}_L)$. Set $\theta' = (\theta_1, \ldots, \theta_{L-1})$ and $\dot{\theta}' = (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1})$. We proceed by induction on L.

Base case $L = 1$. For $L = 1$ we have $\Phi_\theta = x * \theta_1$, hence

$$J_\theta \Phi(\dot{\theta}) = J_{\theta_1}(x * \theta_1)(\dot{\theta}_1) = x * \dot{\theta}_1.$$

This vanishes iff $\dot{\theta}_1 = 0$, so the kernel is trivial, consistent with the proposition.

Inductive step. Assume the proposition holds for all networks with $L-1$ layers. We prove it for L layers.

For brevity set

$$J := J_{\theta'} \Phi(\dot{\theta}').$$

Lemma B.1 implies that

$$\dot{\theta} \in \ker J_\theta \Phi \quad \Longleftrightarrow \quad r\big(J \odot \sigma_{r-1}(\Phi_{\theta'})\big) * \theta_L = -\,\sigma_r(\Phi_{\theta'}) * \dot{\theta}_L. \tag{11}$$

Let $\dot{\theta}_L = 0$. Since θ_L is general, so by Corollary 2.21 we get θ_L is not a zero divisor, and hence $J = 0$. The required result follows by the inductive hypothesis.

Assume $\dot{\theta}_L \neq 0$, convolute both sides of (11) with the inverse of θ_L, which exists because θ_L is general, and let $\dot{\tilde{\theta}}_L := \dot{\theta}_L * \theta_L^{-1}$ to get

$$rJ \odot \Phi_{\theta'}^{r-1} \;=\; -\,\Phi_{\theta'}^{r} * \dot{\tilde{\theta}}_L. \tag{12}$$

Since θ' is general and $\dot{\theta}_L \neq 0$, then by Lemma B.2, we have $\dot{\tilde{\theta}}_L = \lambda e$ for some nonzero $\lambda \in \mathbb{K}$. This implies $\dot{\theta}_L = \lambda \theta_L$, and therefore any $\dot{\theta} \in \ker J_\theta \Phi$ must have the form

$$\dot{\theta} = (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1},\ \lambda \theta_L).$$

Observe that the vector

$$(0, \ldots, 0,\ -\tfrac{\lambda}{r}\theta_{L-1},\ \lambda \theta_L)$$

lies in the kernel as well (it is a direct check using the Jacobian composition rule). Subtracting this vector from $\dot{\theta}$ yields

$$(\dot{\theta}_1, \ldots, \dot{\theta}_{L-1} - (-\tfrac{\lambda}{r}\theta_{L-1}),\ 0) \;=\; (\dot{\theta}_1, \ldots, \dot{\theta}_{L-1} + \tfrac{\lambda}{r}\theta_{L-1},\ 0) \in \ker J_\theta \Phi.$$

By Eq. (11) the vector

$$(\dot{\theta}_1, \ldots, \dot{\theta}_{L-1} + \tfrac{\lambda}{r}\theta_{L-1}) \in \ker J_{\theta'} \Phi,$$

and by the inductive hypothesis, it can be written as a linear combination of

$$\big((\theta_1, -r\theta_2, 0, \ldots, 0),\ (0, \theta_2, -r\theta_3, 0, \ldots, 0),\ \ldots,\ (0, \ldots, 0, \theta_{L-2}, -r\theta_{L-1})\big).$$

Adding back the previously subtracted vector $(0, \ldots, 0, -\frac{\lambda}{r}\theta_{L-1}, \lambda\theta_L)$ shows that $\dot{\theta}$ is a linear combination of the $L-1$ vectors stated in the proposition. This completes the induction and the proof.

□